OPERATING SYSTEMS • PROCESS & THREAD

进程与线程
共享与独立的边界

一本面向秋招的操作系统深度复习书。从"一个 exe 是怎么跑起来的"讲起，一路走到地址空间、TCB、上下文切换、用户级线程、协程、fork/exec/wait 与 Windows 的 CreateProcess。每一个关键结论都配有在本机实测的 C++ 程序输出作为证据。

6 部分 · 11 章 · 1 篇速记附录

6 个 Windows 实测案例 + 2 个 Linux 案例

18 张原理插图

2 个交互实验

Linux / Windows 双平台对照

CHAPTER 00

阅读指南

这本书怎么读、主线是什么、每种标记代表什么。

一条主线：共享与独立

进程与线程这一章的知识点看起来很散：概念、状态、PCB、fork、协程……但它们其实都在回答同一个问题：**哪些东西是大家共用的，哪些东西是各自独有的，边界划在哪里，划在那里有什么代价。**

- **进程**是资源的边界：地址空间、打开的文件、权限凭据都以进程为单位分配和隔离。
- **线程**是执行的边界：每个线程有自己的寄存器、栈和调度状态，CPU 调度的对象是线程。
- **协程 / 任务**把执行边界继续往下切，切到了操作系统看不见的用户态。

后面所有内容，包括"选多进程还是多线程""为什么线程崩溃会带走整个进程""fork 之后文件偏移为什么是共享的"，都可以用这条主线推出来。读的时候遇到新知识点，可以先问自己一句：*它属于共享的那一侧，还是独立的那一侧？*

全书路线图

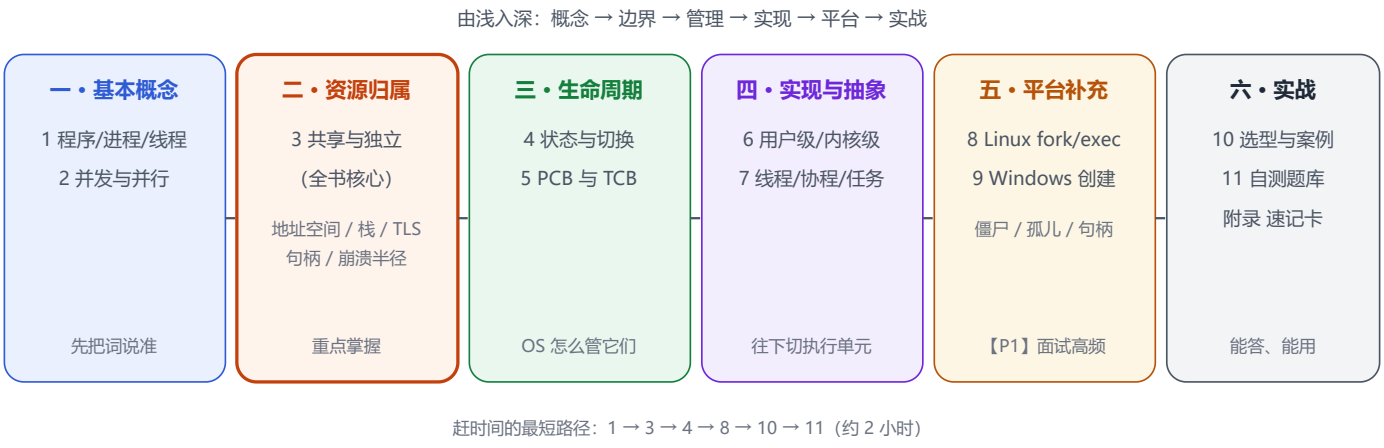


图 0-1 全书结构。第二部分（资源归属）是全书核心，其余各章都会回到它

书中的标记

✅ **面试答法**

可以直接组织成口头回答的要点。

⚠️ **易错点**

常见的错误理解，或者面试官爱挖的坑。

🔬 **深入内核**

Linux / Windows 内核里具体是怎么实现的。第一遍可以跳过，二面、三面前务必读。

🧪 **实测证据**

本书配套程序在本机运行得到的真实输出，用来证明正文结论。

🎮 **游戏开发视角**

这个知识点在游戏引擎 / 游戏服务器中的应用。

关于实测案例

配套源码在本书同目录的 `demos/` 下，共 6 个 Windows 程序（`d1` ~ `d6`）和 2 个 Linux 程序（`demos/linux/`）。Windows 程序全部用 **MSVC 14.44（VS2022）**、`/std:c++20 /O2` 编译，在 Windows 11 x64 上实际运行，书中的输出均为原样粘贴。双击 `demos/build_and_run.bat` 就能在你自己的机器上复现。

程序	证明什么	所在章节
`d1_share`	全局变量、堆同一地址；栈、TLS 每线程一份	第 3 章
`d2_stack_ptr`	线程 B 能通过指针改写线程 A 栈上的数组	第 3 章 / 第 11 章
`d3_isolation`	父子进程同一虚拟地址、不同内容；子进程崩溃不影响父进程；线程崩溃带走整个进程	第 3 章 / 第 9 章
`d4_cost`	线程 vs 进程的创建开销与切换开销	第 4 章 / 第 10 章
`d5_race`	共享内存的代价：数据竞争丢失更新	第 3 章
`d6_fiber_coro`	用户级线程（Fiber）一阻塞全阻塞；C++20 协程单线程并发	第 2 / 6 / 7 章
`linux/fork_exec_wait.c`	fork 返回两次、写时复制、exec 替换映像、waitpid 回收	第 8 章
`linux/zombie_orphan.c`	制造僵尸与孤儿；SIGCHLD 正确回收	第 8 章

> ⚠️ **关于 Linux 案例**
>
> 本机无法运行 Linux 环境，第 8 章中 Linux 程序的输出是**按 Linux 语义给出的典型输出**（PID 等数值会不同），未经本机实测。建议你在 WSL 或云服务器上亲手跑一遍，这本身就是很好的复习。

PART ONE

第一部分　基本概念：先把词说准

程序、进程、线程到底是什么关系？并发和并行差在哪？这两个问题答不准，后面所有内容都会建在沙子上。

CHAPTER 01

程序、进程、线程：从双击一个 exe 说起

程序是静态的文件，进程是资源的容器，线程是执行的流。

本章目标

- 能准确说出程序、进程、线程的定义，以及"资源分配单位"和"调度单位"分别指谁
- 说清楚为什么有了进程还要发明线程
- 理解 Linux 内核眼中"进程和线程都是 task"的含义

1.1 双击 notepad.exe 之后发生了什么

磁盘上的 `notepad.exe` 只是一个文件。它按 PE 格式（Linux 上是 ELF 格式）存放了机器指令、只读常量、全局变量初值、需要导入的 DLL 列表和入口地址。这个文件就是**程序（Program）**，它是静态的、被动的。你不运行它，它就只占一点磁盘空间。

双击之后，操作系统大致做了这几件事：

1. **建立一个新的虚拟地址空间**，并把 exe 文件映射进去：代码段只读可执行，数据段可读写。
2. **分配一张资源表**：句柄表（Linux 上叫文件描述符表），以及安全令牌、当前目录、环境变量等。
3. **创建第一个线程（主线程）**：给它分配一块栈，把指令指针设到程序入口（经过加载器和 C 运行时初始化，最终调用到 `main`）。
4. 把主线程放进**就绪队列**，等调度器选中它，它就开始在 CPU 上执行。

前两步建出来的"一个地址空间 + 一组资源"就是**进程（Process）**；第 3 步建出来的那条"沿着指令一步步往下执行的流"就是**线程（Thread）**。

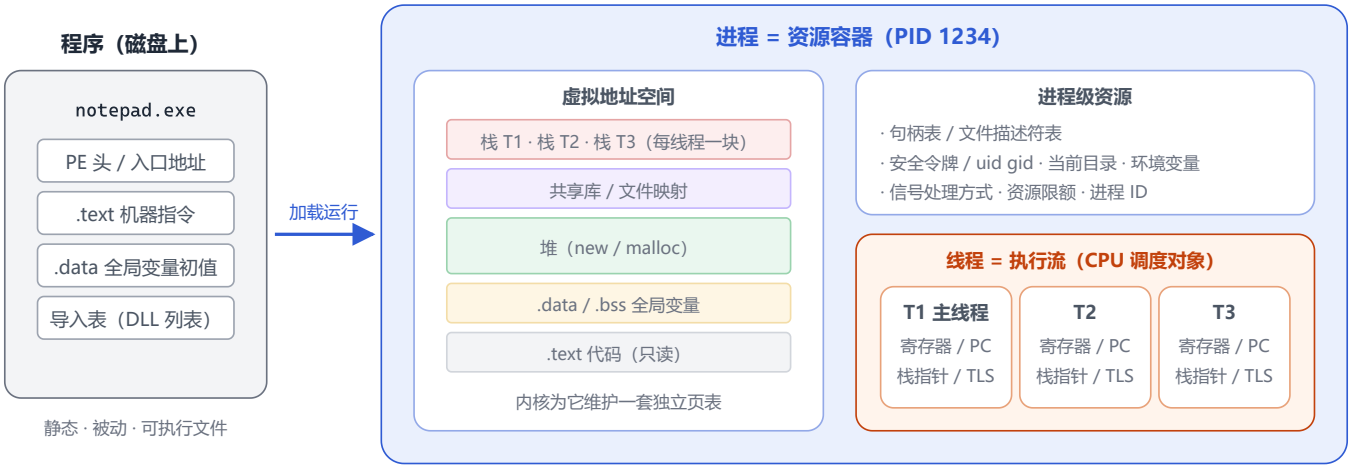


图 1-1 程序是磁盘上的文件；进程是"地址空间 + 资源"组成的容器；线程是容器里真正在 CPU 上跑的执行流

> **一个类比：菜谱、厨房、厨师**
>
> **程序 = 菜谱**：写在纸上，本身不会做出菜。**进程 = 一间开张的厨房**：有自己的灶台、冰箱、食材（地址空间和资源），厨房和厨房之间有墙隔开。**线程 = 厨房里的厨师**：真正干活的是厨师，同一间厨房的厨师共用冰箱和灶台，但每人手上有自己的步骤进度（程序计数器）和自己的案板（栈）。
>
> 按这个类比，同一份菜谱可以同时开好几间厨房（同一个 exe 运行多次就是多个进程）；一间厨房至少要有一个厨师，否则就关门了（进程至少有一个线程）；厨师之间共用冰箱很方便，但两个人同时伸手拿同一块肉就会出问题（数据竞争）。

1.2 进程：资源分配与保护的基本单位

教科书定义：**进程是程序的一次执行实例，是操作系统进行资源分配和保护的基本单位。**这句话里有两个关键词：

资源容器

内存、文件、网络连接、设备等资源都以进程为单位申请和记账。进程退出时，操作系统会把它名下的资源一次性回收，即使程序员忘了 `close` 或 `free`。

保护域

每个进程有**独立的虚拟地址空间**。进程 A 里的地址 `0x7FF6 ...` 和进程 B 里的同一个地址，被各自的页表映射到不同的物理内存，所以 A 无法直接读写 B 的内存。这道墙由 MMU 硬件强制执行。

1.3 线程：CPU 调度的基本单位

线程是进程内的一条独立执行流，是 CPU 调度和分派的基本单位。CPU 只认识"从哪条指令开始执行、寄存器里是什么值、栈在哪"，这套状态就是一个线程。调度器做的事情，就是把某个线程的这套状态装进 CPU，让它跑一会儿，再换下一个。

为什么有了进程还要线程?

早期 Unix 只有进程，一个进程就是一条执行流。后来人们遇到了下面这些需求：

1. **一个程序里有多件事要同时做**：编辑器既要响应键盘，又要后台自动保存、做拼写检查。如果只有一条执行流，保存文件时界面就会卡住。
2. **多核 CPU 出现了**：单个进程只有一条执行流，最多只能用上一个核。
3. **用多进程来做这些事太贵**：每个进程有独立的地址空间，创建时要建页表、复制资源表；进程之间交换数据要走管道、共享内存等 IPC 机制，编程麻烦，开销也大。

于是人们把进程拆成两部分：**资源的部分留在进程里，执行的部分抽出来成为线程。**同一进程的多个线程共享资源，各自独立执行。这样多件事可以同时进行，能用上多个核，而且线程之间交换数据只需要读写同一块内存。

> 🔬 **深入内核：Linux 眼里只有 task**
>
> Linux 内核里没有单独的"线程"数据结构，进程和线程都用 `task_struct` 表示，统称 task。区别只在于创建时用 `clone()` 传了哪些共享标志：
>
> - `fork()` ≈ `clone(SIGCHLD)`：什么都不共享（地址空间写时复制），得到一个新进程。
> - `pthread_create()` ≈ `clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD | ...)`：共享地址空间、文件表、信号处理，得到同一进程里的新线程。
>
> 所以在 Linux 上，每个线程都有自己的 ID（内核叫 pid，用户态叫 **tid**，用 `gettid()` 获取）。同一进程的所有线程组成一个**线程组**，组长的 ID 叫 **tgid**，用户态调用 `getpid()` 拿到的其实就是 tgid。`ps -eLf` 里的 `LWP` 列就是每个线程的 tid。
>
> Windows 的做法相反：内核里有明确分开的进程对象（`EPROCESS`）和线程对象（`ETHREAD`），进程只是容器，本身不执行，**最后一个线程退出时进程随之终止。**

1.4 三者关系一览

维度	程序	进程	线程
本质	磁盘上的可执行文件	运行中的程序实例，资源容器	进程内的执行流
静态 / 动态	静态	动态，有生命周期	动态，有生命周期
是谁的基本单位	—	**资源分配**与保护	**CPU 调度**
数量关系	1 个程序可对应 N 个进程	1 个进程含 ≥1 个线程	属于且只属于 1 个进程
拥有的东西	代码与数据的"模板"	地址空间、句柄表、凭据……	寄存器上下文、栈、TLS、调度状态
标识	文件路径	PID	TID

1.5 用工具亲眼看一看

Linux：查看进程与线程

```
# 看某进程有哪些线程：LWP 列就是线程 tid，NLWP 是线程数
$ ps -eLf | grep -E "PID|chrome" | head
# 每个线程在 /proc/<pid>/task/ 下有一个目录
$ ls /proc/$(pgrep -n chrome)/task
# 树形查看父子关系（{} 包起来的是线程）
$ pstree -p $(pgrep -n sshd)
```

Windows 上打开任务管理器的"详细信息"页，右键列标题勾选"线程"，就能看到每个进程的线程数。Chrome 会显示很多个 `chrome.exe`，因为它是多进程架构：浏览器主进程、每个站点的渲染进程、GPU 进程、网络服务进程各自独立。第 10 章会专门分析它为什么这样设计。

⚠️ **易错点**

- **"进程在执行代码"**：严格地说，执行代码的是线程。进程提供执行环境。
- **"线程没有资源"**：线程不*拥有*系统资源（文件、内存映射归进程），但它*有*自己必需的执行状态：寄存器、栈、TLS 和内核里的 TCB。说"线程只拥有少量必不可少的资源"更准确。
- **"一个程序 = 一个进程"**：一个程序可以运行出多个进程（开两个记事本），一个进程也可以通过 `exec` 换成运行另一个程序。

本章小结

- **程序**是静态文件；**进程**是"地址空间 + 资源"，负责资源分配和隔离；**线程**是执行流，负责被 CPU 调度。
- 线程的出现，是为了把"执行"从"资源"里拆出来：多条执行流共享一套资源，既能并发又省开销。
- Linux 内核统一用 `task_struct`，进程和线程的区别只是 `clone` 时共享了多少；Windows 内核中进程对象和线程对象是分开的。

▶ **自测：同一个 exe 双击两次，两个进程的全局变量 `g_count` 地址打印出来一样，它们是同一块内存吗？**

CHAPTER 02

并发与并行

并发是一种结构，并行是一种执行方式。

本章目标

- 用一句话区分并发和并行，并能举出"并发但不并行"和"并行但不并发"的例子
- 理解单核上为什么也需要并发
- 会用 Amdahl 定律估算多核加速比

2.1 定义

并发 Concurrency

多个任务在**重叠的时间段**内推进。宏观上同时进行，微观上可以交替执行。它描述的是程序的**结构**：程序被组织成多个可以独立推进的部分。

并行 Parallelism

多个任务在**同一时刻**真正同时执行，需要多个执行部件（多核、多 CPU、GPU 的众多核心）。它描述的是**执行**：物理上同时在算。

Go 语言作者之一 Rob Pike 的说法最常被引用：*"Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once."* 并发是"同时应对很多事"，并行是"同时做很多事"。

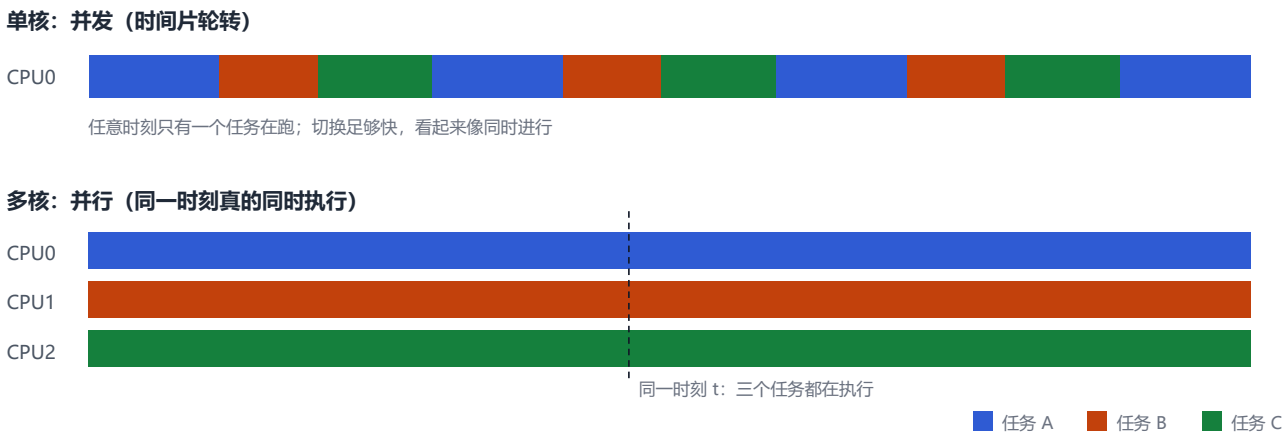


图 2-1 上：单核通过时间片轮转实现并发；下：多核上的并行

2.2 四种组合

并发和并行是两个维度，不是非此即彼：

	并行（多个执行部件同时算）	**不并行（同一时刻只有一个在算）**
并发（多个独立推进的任务）	多线程程序跑在多核上：Web 服务器 8 个工作线程处理请求	单核分时系统；**Node.js / Redis 主线程的事件循环；单线程上的协程调度**
不并发（只有一个逻辑任务）	**SIMD 向量指令**：一条指令同时算 8 个 float；GPU 对一个数组做同一操作	普通的单线程顺序程序

> 🧪 **实测证据：并发但不并行（d6_fiber_coro Part2）**
>
> 两个 C++20 协程 A、B 模拟下载，每下载一块就 `co_await` 让出。全部输出的 tid 都是同一个，说明只有一个线程在跑，但两个任务是**交替推进**的：

d6_fiber_coro.exe（节选）

```
== Part2: C++20 无栈协程，单线程交替推进 ==
  [A] 下载块 1/3  tid=20764
  [B] 下载块 1/2  tid=20764
  [A] 下载块 2/3  tid=20764
```

```
[B] 下载块 2/2  tid=20764
[A] 下载块 3/3  tid=20764
[B] 完成
[A] 完成
```

2.3 单核上为什么也需要并发

如果只有一个核，并发不会让计算变快，因为总计算量不变，切换还有开销。并发的价值在于**让等待和计算重叠**。一次机械硬盘随机读要几毫秒，一次跨机房网络往返要几十毫秒，这段时间 CPU 足够执行上千万条指令。如果程序串行地"发请求 → 干等 → 处理"，CPU 大部分时间都在空转。并发的程序在一个任务等 I/O 时切换去执行别的任务，CPU 利用率和响应速度都会提高。

所以要区分两种负载：

- **I/O 密集型**（Web 服务、网络代理、数据库连接）：瓶颈在等待，并发结构（多线程、事件循环、协程）收益很大，单核也有效。
- **CPU 密集型**（视频编码、物理模拟、寻路、压缩）：瓶颈在计算，只有真正的并行（多核）才能加速。线程数超过核数后基本没有收益，反而增加切换开销。

2.4 并行的天花板：Amdahl 定律

假设程序中可以并行的部分占比为 `p`，用 `n` 个核，理论加速比为：

$$S(n) = 1 / ((1 - p) + p / n)$$

可并行比例 p	4 核	16 核	无穷多核（上限 1/(1−p)）
50%	1.60×	1.88×	2×
90%	3.08×	6.40×	10×
95%	3.48×	9.14×	20×

结论：**串行部分决定上限。**即使 90% 的代码完美并行，再多的核也最多快 10 倍。实际中串行部分不只是代码逻辑，还包括锁竞争、共享缓存行争用、内存带宽，所以实测加速比通常低于理论值。

🔬 深入：超线程算并行吗

Intel 的超线程（SMT）让一个物理核呈现为两个逻辑核，两个硬件线程**共享同一套执行单元和 L1/L2 缓存**，各自有独立的寄存器组。当一个线程因缓存未命中而停顿时，另一个线程可以用空闲的执行单元。所以它是"部分并行"：两个都在跑计算密集任务时，总吞吐通常只比单线程高 10%～30%，远达不到 2 倍。

🎮 游戏开发视角

UE 的 GameThread 和 RenderThread 是典型的**流水线并行**：第 N 帧在 RenderThread 上生成渲染命令时，GameThread 已经在计算第 N+1 帧的逻辑，两者在不同的核上同时运行。资源的**异步加载**则主要是并发：加载线程大部分时间在等磁盘 I/O，主线程不会因此卡住。分清这两者，才能判断优化方向：前者看 CPU 核心负载，后者看 I/O 等待。

本章小结

- **并发**是程序结构（多个任务重叠推进），**并行**是物理执行（同一时刻多个部件在算）。并发不一定并行，并行也不一定是并发。
- 单核上的并发能让 I/O 等待与计算重叠；CPU 密集型任务只有并行能加速。
- Amdahl 定律：加速比上限是 `1/(1-p)`，串行部分（包括锁竞争）决定天花板。

▶ **自测：Redis 6.0 之前的命令处理是单线程的，它是并发的吗？**

PART TWO

第二部分　资源归属：共享与独立的边界

本书最重要的一章。弄清进程拥有什么、线程共享什么、线程各自维护什么，后面的状态切换、选型、fork 语义都能推导出来。

CHAPTER 03

进程拥有什么，线程共享什么，线程独占什么

共享的是资源，独立的是执行状态。但"独立"在很多时候只是约定，而不是硬件保护。

本章目标

- 完整列出进程级资源和线程级状态，并说明每一项为什么放在那一侧
- 用实测输出证明：全局变量和堆是共享的，栈和 TLS 是每线程一份
- 理解"栈私有"不等于"别人访问不到"，以及由此带来的风险
- 理解崩溃半径：为什么一个线程崩溃会让整个进程退出

3.1 一张图看清边界



图 3-1 进程与线程的资源边界。注意虚线框：各线程的栈也位于共享地址空间内

判断规则：为什么这么划分

不需要死记表格，用一条规则推导即可：

> ✅ **推导规则**
>
> **如果两条执行流同时使用某样东西而不冲突，它就可以共享；如果它记录的是"这条执行流自己执行到哪里了"，它就必须独立。**

- **PC 和寄存器**：两个线程执行到不同的指令，算着不同的中间值，只能各有一份。
- **栈**：栈保存函数调用链（返回地址、局部变量）。T1 在 `main → load → parse`，T2 在 `main → render`，两条调用链不同，用同一个栈会互相覆盖。
- **errno**：T1 的 `open` 失败设置了 errno，T2 的 `read` 成功不能把它清零。所以现代 C 库把 errno 实现为线程局部变量（glibc 中是宏 `(*__errno_location())`）。Windows 的 `GetLastError()` 值存放在 TEB 中，同样是每线程一份。
- **文件表**：T1 打开的文件交给 T2 读取，是很常见的需求。共享是合理的。

Microsoft Learn 的官方表述

Windows 文档*About Processes and Threads* 对这个边界有一段很精确的描述，这里转述其要点：

- 进程提供执行程序所需的资源：**虚拟地址空间、可执行代码、指向系统对象的打开句柄、安全上下文、唯一的进程标识符、环境变量、优先级类、最小和最大工作集大小**，以及至少一个执行线程。
- 线程是进程中可以被调度执行的实体。**一个进程的所有线程共享其虚拟地址空间和系统资源。**此外，每个线程维护**异常处理程序、调度优先级、线程局部存储、唯一的线程标识符**，以及系统用来保存线程上下文的一组结构，直到线程被调度。
- 线程上下文包括线程的**机器寄存器集合、内核栈、线程环境块（TEB），以及位于所属进程地址空间中的用户栈。**

最后一句值得仔细读：**用户栈"位于所属进程的地址空间中"**。这正是 3.4 节要讨论的问题的答案。另外请注意，每个线程实际上有**两个栈**：用户态执行时用用户栈，陷入内核（系统调用、中断）后用内核栈。内核栈位于内核地址空间，用户代码无法访问。

3.2 完整对照表

项目	归属	说明
代码段	**进程共享**	只读，所有线程执行同一份代码
全局 / 静态变量	**进程共享**	多线程同时写需要同步。函数内 `static` 局部变量同样是共享的
堆	**进程共享**	一个线程 `new` 的对象可以交给另一个线程 `delete`。分配器内部通常有每线程缓存（如 tcmalloc、jemalloc）来减少锁竞争，但这是性能优化，语义上堆仍是共享的
文件描述符 / 句柄	**进程共享**	T1 `close(fd)` 后，T2 再用这个 fd 会出错；更糟的是 fd 号可能被马上复用，T2 写进了别的文件
当前工作目录	**进程共享**	一个线程 `chdir` 会影响所有线程的相对路径，所以多线程程序应该避免 `chdir`
信号处理函数（Linux）	**进程共享**	`sigaction` 设置的 handler 对整个进程生效
信号屏蔽字（Linux）	**线程独立**	用 `pthread_sigmask` 设置。常见做法：所有线程屏蔽信号，由一个专门的线程 `sigwait` 统一处理
PC、SP、通用寄存器	**线程独立**	切换时保存到 TCB / 内核栈
用户栈	**独立但可达**	每线程一块独立区域，但位于共享地址空间中，拿到指针就能访问
内核栈	**线程独立**	位于内核空间，用户态不可见（Linux x86-64 上一般为 16KB）
TLS（`thread_local`）	**独立但可达**	每线程一份副本。取它的地址交给别的线程，仍然可以访问
errno / GetLastError	**线程独立**	通过 TLS / TEB 实现

调度优先级、状态、亲和性	**线程独立**	调度器以线程为单位调度

3.3 实测：谁共享，谁独立

`d1_share.cpp` 中，主线程 `new` 了一个 int，把指针交给 3 个工作线程。每个线程打印：全局变量地址、堆指针、自己栈上局部变量的地址、`thread_local` 变量的地址，以及用 `GetCurrentThreadStackLimits` 查到的本线程栈范围。

d1_share.cpp（核心部分）

```
int g_global = 100;                 // 数据段：全进程共享
thread_local int t_tls = 0;         // 线程局部存储：每个线程一份

void worker(int id, int* heapPtr)
{
    int local = id * 10;            // 栈：每个线程自己的栈
    t_tls = id;                     // 只改到“自己那一份”
    ULONG_PTR lo = 0, hi = 0;
    GetCurrentThreadStackLimits(&lo, &hi);   // 本线程栈的地址范围
    printf("[T%d tid=%5lu] &g_global=%p heap=%p &local=%p &t_tls=%p(t_tls=%d)\n"  ...);
}
```

d1_share.exe —— 实测输出（Windows 11 x64，MSVC 14.44 /O2）

```
pid=69704  main tid=23692
[T3 tid=67952] &g_global=00007FF643858000  heap=0000018132781990  &local=0000000420DFFC20  &t_t
ls=00000181327B0394(t_tls=3)
              stack=[0000000420D00000, 0000000420E00000)  size=1024 KB
[T2 tid=76768] &g_global=00007FF643858000  heap=0000018132781990  &local=0000000420CFF8E0  &t_t
ls=00000181327C82F4(t_tls=2)
              stack=[0000000420C00000, 0000000420D00000)  size=1024 KB
[T1 tid= 6292] &g_global=00007FF643858000  heap=0000018132781990  &local=0000000420BFFB40  &t_t
ls=00000181327926F4(t_tls=1)
              stack=[0000000420B00000, 0000000420C00000)  size=1024 KB
main: t_tls=0 (主线程自己的 TLS 未被子线程改动)
```

逐项解读：

1. **全局变量和堆：三个线程地址完全相同**（绿色）。它们访问的是同一块内存。

2. **局部变量：三个线程地址各不相同**，并且分别落在各自的栈范围内。T1 的栈是 `[ ... 420B00000, ... 420C00000)`，T2 紧挨着是 `[ ... 420C00000, ... 420D00000)`，T3 是 `[ ... 420D00000, ... 420E00000)`。三块栈**在同一个地址空间里首尾相连**，每块正好 1024 KB。
3. **thread_local：每线程一个不同的地址，值也各不相同**。主线程最后打印 `t_tls=0`，说明子线程写的是它们自己的副本。
4. 线程是**乱序**完成的（T3 先打印）。线程的执行顺序由调度器决定，不能依赖创建顺序。

> 🔬 **深入：栈多大？TLS 在哪？**
>
> - **Windows**：线程栈默认**保留 1MB** 虚拟地址（由链接选项 `/STACK` 或 `CreateThread` 参数决定），但只**提交**很少的几页，栈顶下方有一个**保护页（guard page）**，访问到它时触发异常，系统再提交下一页并把保护页下移。用完 1MB 就会抛出 `STATUS_STACK_OVERFLOW`。上面输出的 `size=1024 KB` 就是保留大小。
> - **Linux**：主线程栈大小由 `ulimit -s` 决定（通常 8MB），可动态向下增长；glibc 创建的线程栈默认也取这个值（8MB），用 `mmap` 分配，并在底部放一个不可访问的**保护区**。可用 `pthread_attr_setstacksize` 调整。
> - **TLS 的实现**：x86-64 上，Linux 用 **FS 段寄存器**指向当前线程的 TCB（其中有 TLS 块），Windows 用 **GS 段寄存器**指向当前线程的 TEB。线程切换时内核会切换这个寄存器的基址，所以同一条指令 `mov eax, fs:[offset]` 在不同线程里读到不同的变量。上面输出中 TLS 的地址落在堆附近的区域（`0x0000018 ...`），因为 MSVC 的 `thread_local` 块是为每个线程动态分配的。

3.4 栈私有 ≠ 别人访问不到

这是本章的重点问题。线程有自己的栈，指的是**每个线程有一块专门给自己用的内存区域来保存调用链**。但这块区域在进程的地址空间里，而同一进程的所有线程用的是**同一套页表**。MMU 只检查"这一页在当前页表里是否映射、是否可写"，它不知道也不关心当前是哪个线程在访问。

所以只要线程 B 拿到了指向线程 A 栈上数据的指针，它就可以读写那块数据。`d2_stack_ptr.cpp` 验证了这一点：

d2_stack_ptr.cpp（核心部分）

```cpp
std::thread A([&] {
    int buf[4] = {1, 2, 3, 4};          // 位于 A 线程的栈上
    shared = buf;                       // 把栈地址交给别的线程
    ready.count_down();
    modified.wait();                    // 等 B 改完（A 的栈帧此时仍然存活）
    printf("[A] 被 B 改写后的 buf = {%d, %d, %d, %d}\n", buf[0], buf[1], buf[2], buf[3]);
});
std::thread B([&] {
    ready.wait();
    for (int i = 0; i < 4; ++i) shared[i] *= 100;    // 跨线程直接写别人的栈
});
```

d2_stack_ptr.exe —— 实测输出

```
[A] buf=00000021219FF7F0  A的栈范围=[0000002121900000, 0000002121A00000)
[B] 我的栈范围=[0000002121A00000, 0000002121B00000)，但我拿着指针 00000021219FF7F0
[B] 写入成功：同一进程 = 同一地址空间，硬件不拦截
[A] 被 B 改写后的 buf = {100, 200, 300, 400}
结论：栈“私有”只是约定（每个线程各用各的区域），不是硬件隔离。
```

B 的栈范围是 `[ ... 2121A00000, ... 2121B00000)`，它手里的指针 `... 21219FF7F0` 明显落在 A 的栈范围内，写入照样成功。

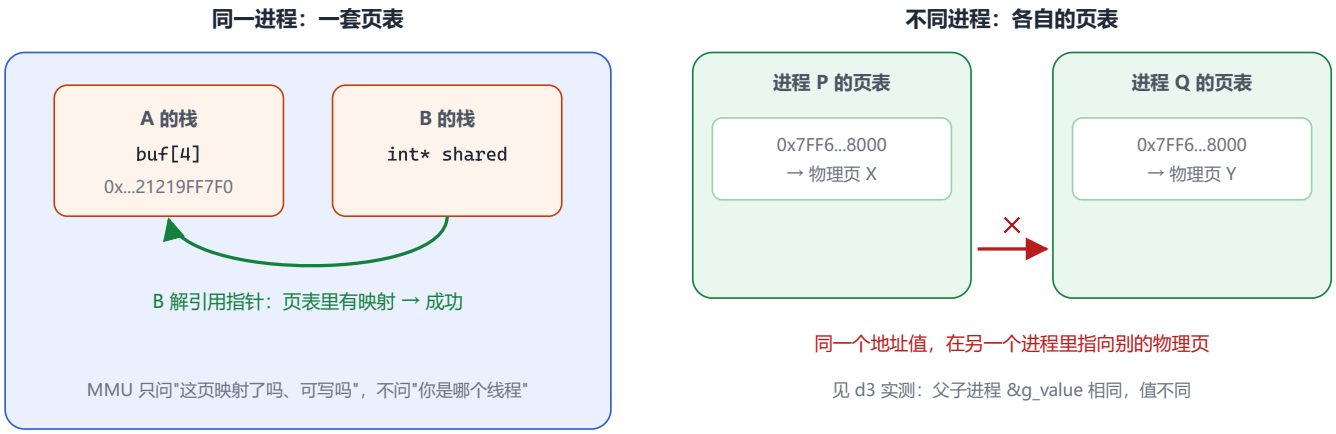


图 3-2 线程之间只有"约定的边界"，进程之间才有"硬件强制的边界"

那为什么还说栈是私有的？三层含义

1. **空间上私有**：每个线程分到一段专属的地址区间，编译器生成的 `push` / `call` /局部变量访问只会使用自己的 SP 指向的区域，正常情况下不会碰到别人的栈。
2. **生命周期私有**：栈上对象的生命周期由**拥有它的线程**的函数调用决定。函数返回，栈帧就失效；线程退出，整块栈都会被释放。其他线程**无权也无法**延长它的寿命。
3. **默认不共享**：如果没有人主动把栈地址交出去，别的线程就没有办法找到它（没有名字，地址也不固定）。

所以，**跨线程访问栈上数据在语法和硬件上都是允许的，难点在生命周期**：

⚠️ 易错点：悬垂的栈指针（最常见的真实 Bug）

错误示范：把局部变量的地址交给异步任务

```
void StartLoad(ThreadPool& pool)
{
    char path[260] = "data/level1.pak";
    pool.Submit([&path] {              // 按引用捕获了栈上的数组
        LoadFile(path);                // 工作线程执行时，StartLoad 很可能已经返回
    });
}   // ← path 所在栈帧在这里失效，之后这块内存会被下一次函数调用覆盖
```

这种 Bug 很难复现：工作线程启动得快时程序正常，慢一点就读到垃圾数据，Debug 和 Release 表现还不一样。**修复方法是让数据的生命周期覆盖所有使用者**：按值捕获（`[path = std::string(path)]`），或者放到堆上用 `std::shared_ptr` 管理，或者保证在栈帧返回前 `join` /等待任务完成（d2 就是用 `latch` 保证了 A 的栈帧在 B 写完之前一直存活）。

✅ 合法而常见的跨线程访问栈的用法

- **`std::thread` + 引用传参 + 在同一作用域 `join`**：主线程栈上的数组交给工作线程处理，处理完 `join`，生命周期是安全的。`std::jthread` 在析构时自动 `join`，更不容易出错。
- **同步调用**：线程 A 把栈上的请求结构体地址放入队列，然后阻塞等待工作线程处理完成，再继续执行。很多 RPC 框架和 Windows 的 `SendMessage` 都采用这种模式。
- **OpenMP / parallel_for**：并行循环访问外层函数的局部数组，因为外层函数会等所有迭代结束才返回。

🔬 **深入：栈溢出会写坏别人的栈吗**

d1 的输出显示，各线程的栈在地址上首尾相连。理论上，T2 的栈向下溢出就会进入 T1 栈的高地址部分。实际上中间隔着**保护页**：Windows 的 guard page 和 Linux 的 guard area 都不可访问，溢出时会先触发异常（Windows `STATUS_STACK_OVERFLOW`，Linux `SIGSEGV`），程序直接崩溃。但如果在栈上开了一个比保护页还大的局部数组（例如 `char buf[64*1024]`）并且从远端开始写，就可能**跳过保护页**直接写到相邻区域。因此编译器会在大栈帧函数中插入**栈探测**（MSVC 的 `__chkstk`，GCC 的 `-fstack-clash-protection`），按页依次触碰，确保一定会碰到保护页。

3.5 共享的代价：数据竞争

共享地址空间让线程间通信几乎零成本，代价是：**多个线程同时读写同一块内存，且至少有一个是写，又没有同步，就构成数据竞争。**在 C++ 标准中，数据竞争是**未定义行为。**

最经典的例子是 `counter++`。它看起来是一条语句，实际是三步：从内存读到寄存器 → 寄存器加 1 → 写回内存。两个线程交错执行就会丢失更新：

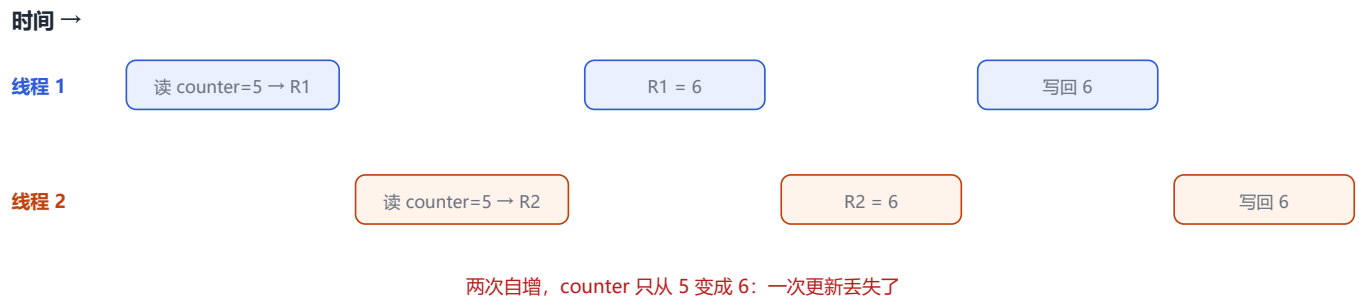


图 3-3 读-改-写被打断导致的丢失更新

```
d5_race.exe —— 4 线程各自增 100 万次，实测输出

普通变量 ++ (round) 结果=  1180103  期望=4000000  耗时=   9.2 ms
普通变量 ++ (round) 结果=  3390547  期望=4000000  耗时=   2.8 ms
普通变量 ++ (round) 结果=  1737623  期望=4000000  耗时=   1.7 ms
std::atomic fetch_add  结果=  4000000  期望=4000000  耗时=  21.5 ms
std::mutex 加锁        结果=  4000000  期望=4000000  耗时=  78.1 ms
```

三点观察：

1. 普通变量每次结果都不一样，最少的一次只有期望值的 30%。**结果不确定**正是数据竞争的特征。
2. 代码中的 `volatile` 只是阻止编译器把循环优化成一条加法，**它不提供原子性**。这是 C++ 面试的常见陷阱（Java 的 volatile 语义不同，有可见性和有序性保证，但同样不保证复合操作的原子性）。
3. 正确的版本更慢：`atomic` 慢约 2～10 倍，`mutex` 更慢。原因是多个核争抢同一条**缓存行**，每次写都要通过缓存一致性协议使其他核的副本失效。**共享可写数据本身就有成本**，所以高性能设计会尽量让每个线程写自己的数据（例如每线程一个局部计数器，最后汇总）。

3.6 进程之间：硬件强制的隔离

`d3_isolation.cpp` 用 `CreateProcess` 启动自身的另一个实例作为子进程，比较两边的全局变量：

d3_isolation.exe —— 实测输出（第一部分）

```
[parent pid=66436] &g_value=00007FF79F558000  g_value=1
  [child pid=75908] &g_value=00007FF79F558000  改成 999
[parent] 子进程退出码=7，父进程 g_value 仍然=1
```

两个进程中 `g_value` 的**虚拟地址完全相同**，子进程改成 999，父进程仍然是 1。同一个虚拟地址，在两套页表中映射到不同的物理页。这就是图 3-2 右半部分。

进程之间要交换数据，必须借助内核提供的 **IPC（进程间通信）**机制：

机制	Linux	Windows	特点
管道	`pipe`、FIFO	匿名管道、命名管道	字节流，简单，适合父子进程
共享内存	`shm_open`+`mmap`、System V shm	`CreateFileMapping`+`MapViewOfFile`	**最快**（零拷贝），但需要自己同步，相当于把一部分页"故意"映射到多个进程
消息队列	POSIX mq、System V msg	窗口消息、邮槽、ALPC（内部）	有消息边界
套接字	Unix domain socket、TCP	TCP、AF_UNIX（Win10+）	通用，可跨机器
信号 / 事件	signal、eventfd	命名 Event、Semaphore	只传递通知，不传数据

3.7 崩溃半径：线程崩溃，整个进程陪葬

共享地址空间的另一面是**故障也共享。** `d3_isolation` 的后两个实验：

d3_isolation.exe -- 实测输出（第二、三部分）

```
[parent] 子进程空指针崩溃，退出码=0xC0000005 (STATUS_ACCESS_VIOLATION)，父进程安然无恙

  [threadcrash] 主线程仍在工作 0
  [threadcrash] 主线程仍在工作 1
  [threadcrash] 主线程仍在工作 2
[parent] 子进程的“某个工作线程”崩溃 → 整个子进程被终止，退出码=0xC0000005
```

- **子进程崩溃，父进程不受影响**：进程边界把故障挡在了外面。
- 在 `threadcrash` 模式下，子进程的主线程一直在正常打印，100ms 后一个**工作线程**解引用空指针，**整个子进程（包括正常工作的主线程）立刻被终止。**

为什么不能只杀掉出错的那个线程?

因为**没有办法知道它破坏了什么。**出错的线程可能已经写坏了堆上的数据结构，也可能正持有一把锁（如果只杀掉它，这把锁永远不会被释放，其他线程等这把锁时会死锁），还可能正在修改一个被所有线程共享的全局状态，修改到一半。由于地址空间是共享的，这些破坏对所有线程都可见，继续运行的结果不可预测。所以操作系统的默认策略是终止整个进程：

- **Linux**：`SIGSEGV` 是发给**线程组（整个进程）**的致命信号，默认动作是终止整个进程并生成 core dump。
- **Windows**：未处理的结构化异常最终交给系统的未处理异常过滤器，默认调用 WER（错误报告）并终止进程。

> 🎮 **游戏开发视角**
>
> 这就是为什么游戏客户端里一个线程（比如音频线程、资源加载线程）的空指针，会让整个游戏闪退，并在崩溃报告里看到崩溃线程不是 GameThread。也是为什么 Chrome、VS Code、Unreal Editor 的部分功能（Shader 编译、插件宿主、渲染进程）被拆到**独立进程**：它们崩溃时，主进程可以显示"页面崩溃，点击重新加载"或重启子进程，而不是整体退出。UE 的 `ShaderCompileWorker.exe` 就是一个例子。

> 🍼 **交互实验 1 • 归属判断训练**

下面每一项是"同一进程内所有线程共享"，还是"每个线程独立维护"？点击按钮作答，会给出解释。

1. Windows GetLastError() 的值

进程共享　线程独立　下一题 →

点击"进程共享"或"线程独立"开始。

本章小结

- **进程共享**：地址空间（代码、全局变量、堆、映射、*以及所有线程的栈所在的区域*）、句柄 / fd 表、凭据、工作目录、信号处置方式。
- **线程独立**：TID、寄存器上下文（PC/SP）、用户栈与内核栈、TLS（errno）、调度状态与优先级、信号屏蔽字 / 异常处理链。
- 线程之间的"独立"是**约定**：栈和 TLS 只要拿到地址就能被别的线程访问，难点在生命周期。进程之间的隔离由 **MMU + 独立页表**强制执行。
- 共享带来零成本通信，也带来数据竞争和"一个线程崩溃，整个进程退出"。

PART THREE

第三部分　生命周期与管理结构

操作系统怎么知道一个进程/线程处于什么状态？在什么事件下切换状态？靠什么数据结构记住它的一切？

CHAPTER 04

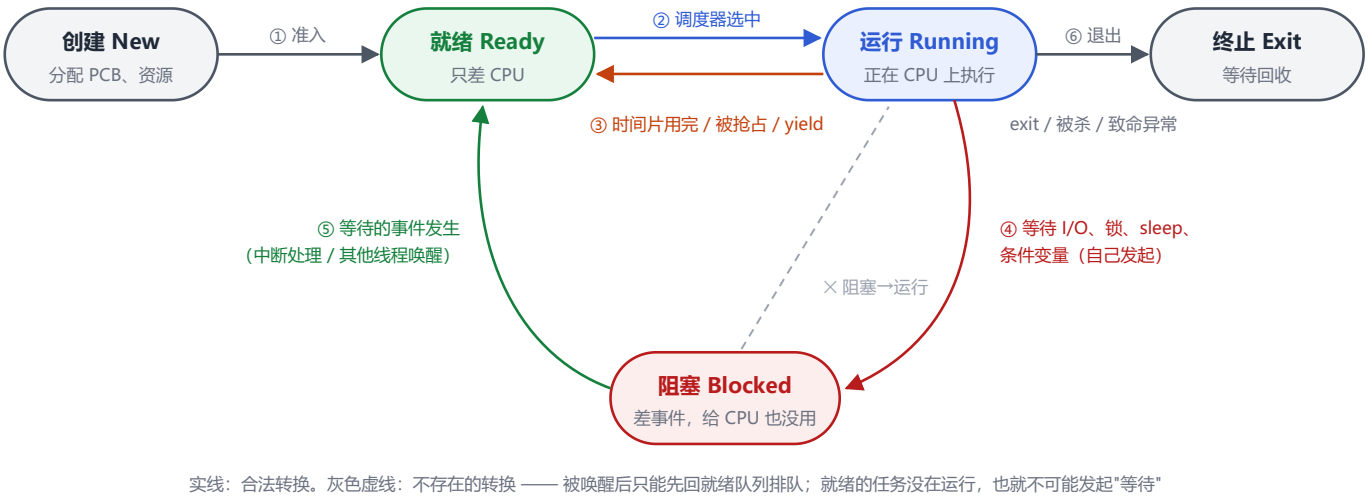
生命周期与状态转换

五个状态、六条合法的边，以及每条边背后由谁触发。

本章目标

- 画出五状态模型，说出每条转换的触发事件，以及哪些转换不存在、为什么不存在
- 把教科书模型对应到 Linux 的 `R/S/D/T/Z` 和 Windows 的线程状态
- 理解上下文切换做了什么，进程切换为什么比线程切换贵

4.1 五状态模型



实线：合法转换。灰色虚线：不存在的转换 —— 被唤醒后只能先回就绪队列排队；就绪的任务没在运行，也就不可能发起"等待"

图 4-1 五状态模型。注意 ③ 和 ④ 的区别：③ 是被动让出 CPU 但随时可以再跑，④ 是自己发起等待，事件不来就不能跑

转换	触发者	典型例子
① 创建 → 就绪	内核完成初始化	`fork` / `CreateProcess` 返回前，新任务进入就绪队列
② 就绪 → 运行	**调度器**	当前任务让出 CPU，调度器从就绪队列选出下一个
③ 运行 → 就绪	时钟中断 / 调度器	时间片用完；更高优先级的任务被唤醒并抢占；调用 `sched_yield` / `SwitchToThread`
④ 运行 → 阻塞	**任务自己**（经系统调用）	`read` 磁盘、`recv` 网络、`mutex.lock()` 没抢到、`sleep`、`cv.wait`
⑤ 阻塞 → 就绪	中断处理程序 / 其他任务	磁盘 DMA 完成触发中断；持锁线程 `unlock` 唤醒等待者；定时器到期
⑥ 运行 → 终止	任务自己 / 内核	`return` 出 `main`；`exit`；收到 `SIGKILL`；访问违例

✅ **面试答法：为什么没有"阻塞 → 运行"和"就绪 → 阻塞"？**

阻塞 → 运行：事件发生只说明它*可以*运行了，但 CPU 可能正被别人占用，必须先进入就绪队列，由调度器统一决定谁上 CPU。（有些内核会在唤醒时检查优先级，若被唤醒者更高就立即触发抢占，但逻辑上仍是"先就绪、再被调度"。）

就绪 → 阻塞：阻塞是任务*自己*执行了一个"等待"操作（系统调用）的结果，而就绪的任务根本没有在执行，不可能发起等待。

🧪 **交互实验 2 · 状态机模拟器**

模拟一个进程从创建到终止的过程。只有在当前状态下合法的事件按钮才可点击，对照上图体会每条边。

当前状态： **创建**

初始化完成，准入 | 调度器选中 | 时间片用完（时钟中断） | 被高优先级线程抢占

调用 read() 等待磁盘 | mutex.lock() 没抢到 | 磁盘中断：数据到达 / 锁被释放 | return / exit()

访问空指针（致命异常）　　↺ 重新开始

新进程：`fork()` / `CreateProcess()`

4.2 一次 `read()` 经历的完整状态变化

把抽象模型放到具体场景里：一个线程调用 `read(fd, buf, 4096)` 读一个不在页缓存里的文件。

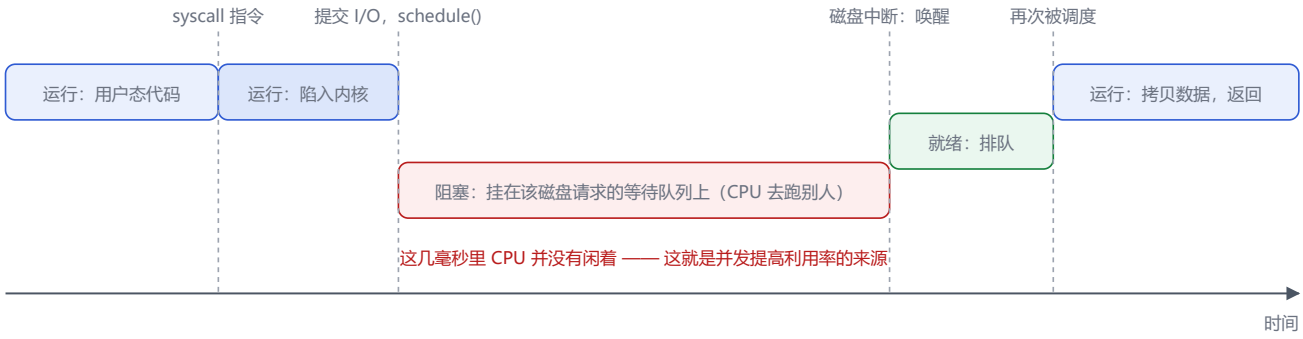


图 4-2 一次阻塞式 read 的状态时间线。注意"陷入内核"本身不是状态变化，线程仍然是运行态，只是从用户态切到了内核态

> ⚠️ **易错点：用户态/内核态 ≠ 运行/阻塞**
>
> "用户态、内核态"说的是 CPU 的**特权级**；"运行、就绪、阻塞"说的是**调度状态**。一个线程执行系统调用时进入内核态，但它仍然是"运行"状态；只有在内核里发现需要等待并调用调度函数让出 CPU 时，它才变为"阻塞"。

4.3 扩展：挂起状态（七状态模型）

当内存紧张时，操作系统可能把某些进程整个换出到磁盘，这时就多出两个状态：**就绪挂起**（在磁盘上，换入就能跑）和**阻塞挂起**（在磁盘上，还在等事件）。现代通用操作系统以**页**为单位换出内存，很少整进程换出，所以这个模型更多出现在教材和考研题中。了解即可：**挂起关注的是"在不在内存里"，阻塞关注的是"在不在等事件"，两者正交。**

4.4 真实系统里的状态

Linux：`ps` 的 STAT 列

STAT	内核状态	对应教科书	说明
`R`	TASK_RUNNING	就绪 + 运行	**Linux 不区分就绪和运行**，都是 R，区别只在是否正在某个 CPU 上
`S`	TASK_INTERRUPTIBLE	阻塞	可中断睡眠：信号可以把它唤醒（例如 `sleep`、等待 socket）
`D`	TASK_UNINTERRUPTIBLE	阻塞	不可中断睡眠：通常在等磁盘 / NFS I/O。`kill -9` **也杀不掉**，大量 D 状态进程通常说明存储出了问题
`T` / `t`	TASK_STOPPED / TRACED	（挂起）	被 `SIGSTOP` / `Ctrl+Z` 暂停，或正被调试器跟踪
`Z`	EXIT_ZOMBIE	终止	已退出但父进程尚未 `wait`，第 8 章详述
`X`	EXIT_DEAD	终止	正在被彻底回收，几乎看不到

Linux 的平均负载（load average）统计的是 **R + D** 状态的任务数。所以负载高不一定是 CPU 忙，也可能是大量任务卡在磁盘 I/O 上。

Windows：线程状态

Windows 调度的对象是线程，进程本身没有运行状态。线程状态（`KTHREAD_STATE`）比教科书更细：**Initialized**（创建中）→ **Ready**（就绪）→ **Deferred Ready**（已选定处理器，等待被分派）→ **Standby**（已被选为某处理器下一个要运行的线程）→ **Running** → **Waiting**（阻塞）→ **Terminated**。另外还有 **Transition**（就绪但内核栈被换出内存）。可以在 Process Explorer 中查看线程属性观察这些状态。

4.5 上下文切换：状态转换的代价

每次"运行 → 其他状态 → 运行"，CPU 都要从一个线程换到另一个线程，这叫**上下文切换**。它包括：

1. **进入内核**：通过时钟中断、系统调用或异常。
2. **保存当前线程的上下文**：通用寄存器、PC、SP、标志寄存器，必要时还有浮点 / SIMD 寄存器（AVX-512 状态可达 2KB 以上），保存到它的内核栈和 TCB 中。
3. **调度器选择下一个线程。**
4. **如果下一个线程属于另一个进程**：切换页表（x86 上写 `CR3` 寄存器）。
5. **恢复下一个线程的上下文**，切换内核栈、FS/GS 基址（TLS），返回用户态。



图 4-3 线程切换与进程切换的差别在于地址空间：要不要换页表，TLB 和缓存还能不能用

> 🧪 **实测证据：d4_cost 的 ping-pong 测试**
>
> 把进程绑定到同一个 CPU 核上，两个执行流用 Event 来回唤醒对方 10 万次，每一次交互必然发生一次上下文切换。测得的单次时间包含了 `SetEvent` / `WaitForSingleObject` 两次系统调用的开销。

```
d4_cost.exe —— 实测输出（同核 ping-pong）

单次切换(含事件系统调用)  线程间：0.68 us    进程间：1.50 us
```

跨进程切换是线程切换的约 2.2 倍。需要注意，这个微基准中两个进程的工作集都极小，**没有体现缓存和 TLB 被污染的间接开销。**在真实程序中，切换后需要重新加载的缓存行和 TLB 项越多，差距越大。

🔬 深入：PCID 与 KPTI

早期 x86 写 CR3 会刷新整个 TLB（全局页除外）。现代 CPU 支持 **PCID**（进程上下文标识符），TLB 项带上地址空间标签，切换后旧进程的 TLB 项可以保留，切回来时继续使用。2018 年 Meltdown 漏洞之后，内核引入了 **KPTI**（内核页表隔离）：用户态和内核态使用两套页表，于是**每次系统调用**也要切换 CR3，PCID 在这里同样能大幅降低开销。这也是为什么当年打 Meltdown 补丁后，系统调用密集型的程序（数据库、I/O 密集服务）性能下降明显。

本章小结

- 五状态：创建、就绪、运行、阻塞、终止；六条合法转换。**就绪是"差 CPU"，阻塞是"差事件"。**
- 阻塞由任务自己发起，唤醒由别人完成；唤醒后先回就绪队列，不能直接运行。
- Linux 用 R 同时表示就绪和运行；D 状态不可被信号打断；Z 是僵尸。
- 上下文切换要保存/恢复寄存器；跨进程还要切换页表，导致 TLB 和缓存失效。实测同核切换：线程 0.68μs，进程 1.50μs。

CHAPTER 05

进程控制块与线程控制块

PCB 是进程在内核中的"档案"，TCB 是线程的"档案"。理解它们要记什么，就不用背字段。

本章目标

- 从"操作系统需要对进程做哪些事"推导出 PCB 必须记录的信息
- 说清楚 PCB 和 TCB 的分工，以及它们与第 3 章资源边界的对应关系
- 认识 Linux `task_struct` 与 Windows `EPROCESS/ETHREAD`、`PEB/TEB`

5.1 从需求推导 PCB 的内容

不要背字段。问自己：操作系统要管理一个进程，需要做哪些事？每件事需要记住什么？

操作系统要做的事	因此必须记录	放在 PCB 还是 TCB
区分不同的进程 / 线程	PID、TID、父进程 ID、所属用户	两者都有各自的 ID
暂停一个执行流，以后从原处继续	**CPU 上下文**：PC、SP、通用寄存器、标志寄存器、浮点状态	**TCB**
决定下一个让谁运行	调度状态、优先级、已用 CPU 时间、所在队列的链接指针	**TCB（调度单位是线程）**
把虚拟地址翻译成物理地址	页表基址、内存区域列表（代码 / 堆 / 栈 / 映射）	**PCB**
管理打开的文件和设备	文件描述符表 / 句柄表	**PCB**
检查权限	uid/gid、能力集、访问令牌	**PCB（Windows 线程可临时模拟另一身份）**
维护进程家族、回收退出码	父进程、子进程链表、退出状态	**PCB**
投递信号 / 异步通知	信号处置方式（进程）、待处理信号与屏蔽字（线程）	两者都有
统计和限制资源	CPU 时间、内存用量、资源上限	两者都有

这张表的第三列和第 3 章的资源边界是**一一对应**的：进程共享的东西记在 PCB，线程独立的东西记在 TCB。PCB 可以概括为三类信息：**标识信息、处理器状态信息（保存在 TCB 中）、控制信息（调度、内存、资源、家族、统计）**。

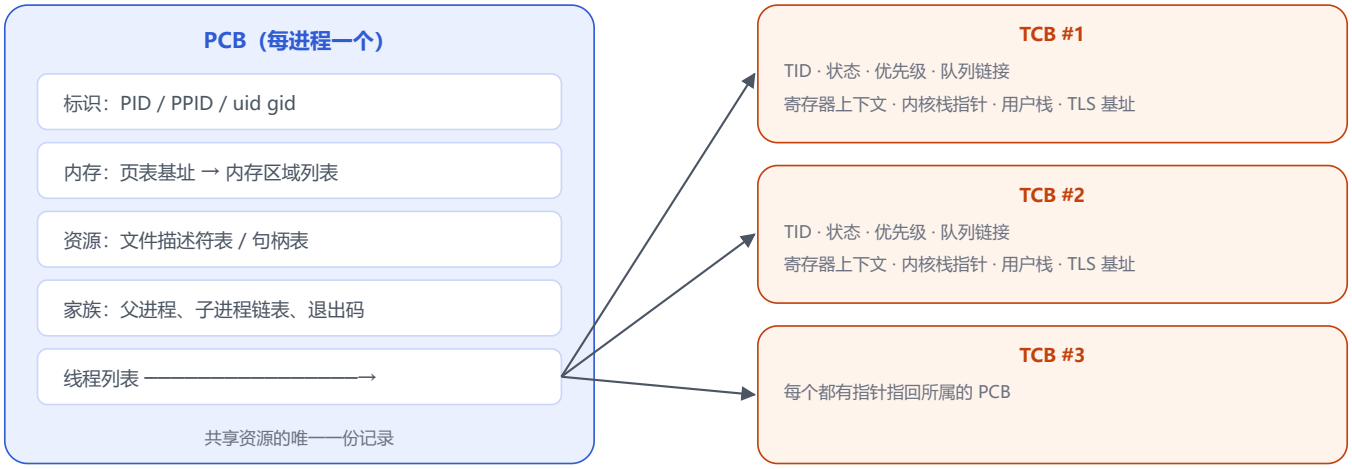


图 5-1 PCB 与 TCB 的一对多关系

5.2 PCB 如何被组织起来

单个 PCB 只是档案，操作系统还要能快速找到"所有就绪的线程""等待某个磁盘请求的线程"。方法是把 TCB 挂到不同的队列 / 数据结构上：

- **就绪队列**：Linux CFS 调度器为每个 CPU 维护一棵按"虚拟运行时间"排序的**红黑树**，每次选最左边（运行得最少）的任务；Linux 6.6 起换成了 EEVDF 调度器，但仍然是每 CPU 一个运行队列。Windows 为每个处理器维护 32 个优先级的就绪队列，配合位图快速找到最高优先级的非空队列。
- **等待队列**：每个可等待的对象（一把锁、一个 socket、一个磁盘请求）都有自己的等待队列。线程阻塞时挂上去，事件发生时被摘下、放回就绪队列。
- **全局索引**：按 PID 查找（Linux 用 IDR/基数树），以及所有进程的链表。

状态转换在实现上就是**把 TCB 从一个队列移到另一个队列**，再修改它的状态字段。

5.3 Linux：`task_struct`

如第 1 章所说，Linux 中进程和线程都由 `task_struct` 表示（定义在 `include/linux/sched.h`，有几百个字段）。它的巧妙之处在于：**共享的资源不直接放在 task_struct 里，而是放在独立的结构体中，用指针引用。**同一进程的线程，指针指向同一个结构体；不同进程的指针指向不同的结构体。

task_struct 极简示意（非真实定义，只保留与本章相关的字段）

```
struct task_struct {
    /* ——— 每个 task 自己的：相当于 TCB ——— */
    unsigned int          __state;      // TASK_RUNNING / TASK_INTERRUPTIBLE / ...
    pid_t                 pid;          // 内核视角的 ID，即用户态的 tid
    pid_t                 tgid;         // 线程组 ID，即用户态 getpid() 的返回值
    int                   prio;         // 动态优先级
    struct sched_entity   se;           // CFS/EEVDF 调度实体：vruntime、红黑树节点……
    void                 *stack;        // 内核栈
    struct thread_struct  thread;       // 体系结构相关的 CPU 上下文（sp、fs 基址……）
    sigset_t              blocked;      // 本线程的信号屏蔽字

    /* ——— 指向共享资源的指针：相当于 PCB 的内容 ——— */
    struct mm_struct      *mm;          // 地址空间：页表 pgd + 内存区域 VMA 列表
    struct files_struct   *files;       // 文件描述符表
    struct fs_struct      *fs;          // 根目录、当前工作目录
    struct signal_struct  *signal;      // 线程组共享的信号状态
    struct sighand_struct *sighand;     // 信号处理函数表
    const struct cred     *cred;        // uid / gid / capabilities

    /* ——— 家族关系 ——— */
    struct task_struct    *real_parent;
    struct list_head       children, sibling;
    int                    exit_code;   // 退出码，僵尸状态时保留给父进程
};
```

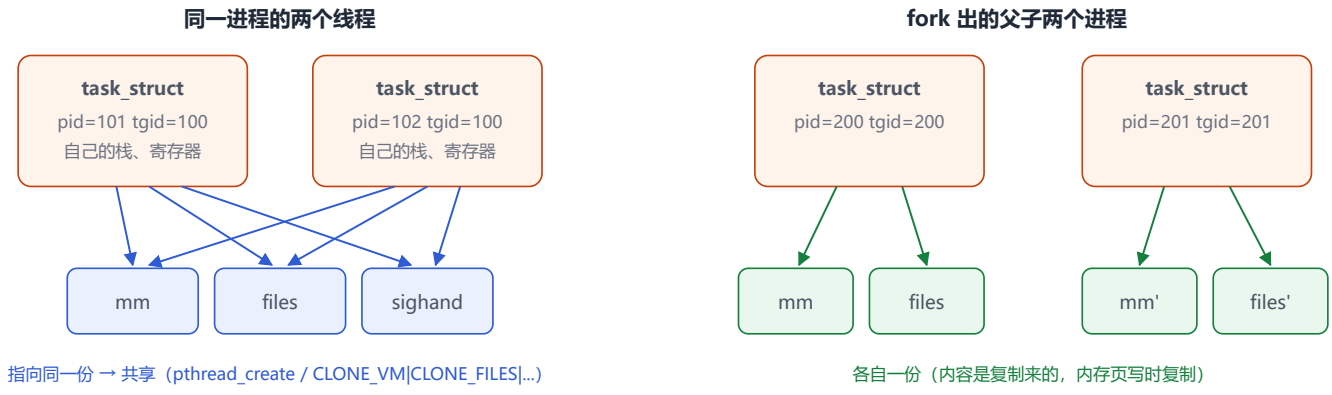


图 5-2 Linux 用"指针是否指向同一个结构体"来表达共享与独立

🔬 **深入：内核线程的 mm 为什么是 NULL**

内核线程（如 `kworker`、`ksoftirqd`）只在内核空间运行，不需要用户地址空间，所以它们的 `mm` 为 `NULL`。调度到内核线程时，内核会"借用"上一个用户进程的页表（记在 `active_mm` 中），因为所有进程页表中的内核部分都是相同的，这样就省掉了一次 CR3 切换。这是一个"不切换地址空间就能省开销"的实例，和第 4 章的分析一致。

5.4 Windows：内核结构 + 用户态结构

结构	位于	作用
`EPROCESS`（内含 `KPROCESS`）	内核空间	进程对象：PID、页表目录基址、句柄表、访问令牌、线程列表、Job 归属……`KPROCESS` 是其中调度相关的部分
`ETHREAD`（内含 `KTHREAD`）	内核空间	线程对象：TID、状态、优先级、内核栈、等待块、上下文……
PEB（进程环境块）	**用户空间**	已加载模块列表、进程堆、命令行、环境变量指针。`kernel32` / `ntdll` 在用户态直接读取它，省去系统调用
TEB（线程环境块）	**用户空间**	栈基址与栈限制、TLS 槽、`LastErrorValue`、SEH 链表头、指向 PEB 的指针。x64 下由 `GS` 寄存器指向

PEB / TEB 的设计说明了一点：**"线程控制信息"不一定都在内核里。**为了性能，那些用户态频繁读写、又不涉及安全的状态（如 `GetLastError` 的值、栈边界）被放在用户空间。d1 中调用的 `GetCurrentThreadStackLimits` 本质上就是从 TEB 中读取栈的上下界，不需要进入内核。

⚠️ **易错点**

面试问"PCB 包含什么"时，不要逐个背 Linux 字段名，而应该按**"标识、处理器状态、调度信息、内存管理、资源清单、家族与统计"**分类回答，再补一句"其中处理器状态和调度信息在多线程系统中下沉到了每个线程的 TCB"。这能体现你理解了 PCB 与 TCB 的分工。

本章小结

- PCB 记录进程的"标识 + 资源 + 控制信息"，TCB 记录线程的"执行上下文 + 调度信息"。两者的划分与第 3 章的资源边界一致。
- 状态转换在实现上就是在就绪队列、等待队列之间移动 TCB。
- Linux 的 `task_struct` 通过指针共享 `mm/files/sighand` 来区分进程和线程；Windows 有 `EPROCESS/ETHREAD` 以及用户态的 `PEB/TEB`。

PART FOUR

第四部分　实现与抽象：执行单元往下切

线程可以由内核实现，也可以完全在用户态实现。再往下切，就得到了协程；再往上抽象，就得到了任务。

CHAPTER 06

用户级线程与内核级线程

关键问题只有一个：内核知不知道这个线程的存在？

本章目标

- 说清楚用户级线程和内核级线程的定义、优缺点，以及根源在哪
- 掌握 1:1、N:1、M:N 三种映射模型，并知道主流系统和语言分别选了哪种
- 用实测证明"用户级线程一个阻塞，全部阻塞"

6.1 定义

用户级线程（ULT）

线程的创建、切换、调度全部由**用户态的线程库**完成。内核完全不知道它们的存在，在内核看来这个进程只有一条执行流（一个内核线程）。

内核级线程（KLT）

线程由**内核**创建和管理，每个线程有内核中的 TCB，由内核调度器调度到 CPU 上。`pthread`（Linux NPTL）、Windows 线程、`std::thread` 都是内核级线程。

6.2 三种映射模型

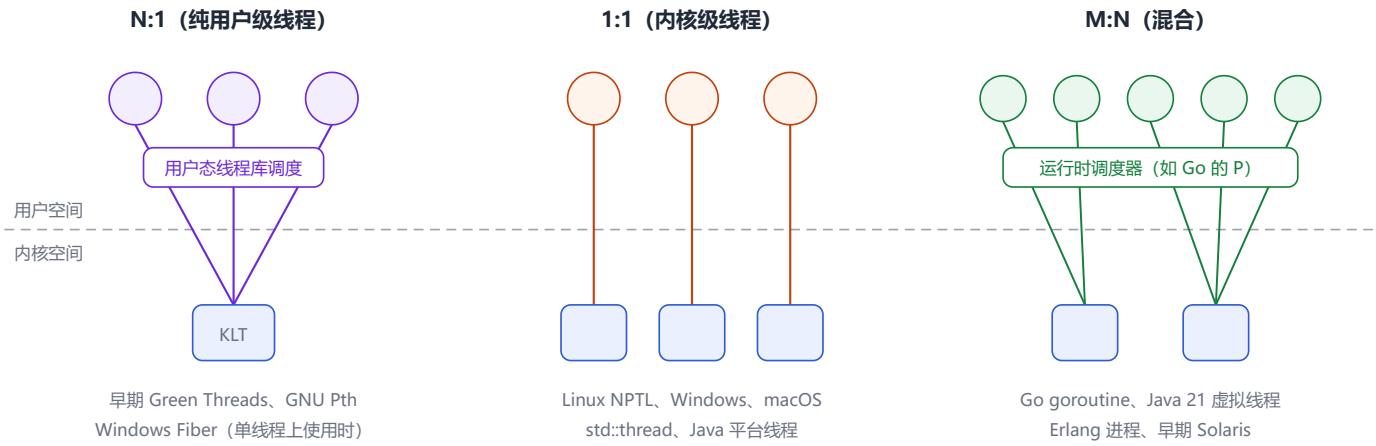


图 6-1 三种映射模型。上方圆圈是用户可见的"线程"，下方方块是内核调度的实体

6.3 优缺点对照，以及根源

维度	用户级线程（N:1）	内核级线程（1:1）	根源
创建 / 切换开销	**极小（几十纳秒，函数调用级别）**	**较大（需陷入内核，约微秒级）**	ULT 切换只是保存几个寄存器、换个栈指针，不进内核
一个线程阻塞	**整个进程都阻塞**	**只阻塞自己**	内核只看见一个执行流，它阻塞了，内核就把整个进程挂起
利用多核	**不能**	**能**	内核只会把这一个执行流放到一个 CPU 上
调度策略	**应用可自定义**	**由内核决定**	ULT 调度器就是应用自己的代码
抢占	**通常不能（协作式）**	**时钟中断强制抢占**	时钟中断由内核处理，用户态库收不到（除非用信号模拟）
数量上限	**可达百万级**	**数千～数万**	KLT 每个都有内核栈和内核数据结构，用户栈默认 1～8MB 虚拟空间

表中所有优缺点都源自同一件事：**内核看不看得见。**看不见，就不用进内核，所以快；看不见，内核也就没法帮它调度、没法帮它在别的核上运行、没法在它阻塞时切换到同进程的其他线程。

> 💉 **实测证据：Windows Fiber 一个阻塞，全部阻塞（d6_fiber_coro Part1）**
>
> Windows 的 **Fiber（纤程）**是操作系统提供的用户级线程原语：每个 Fiber 有自己的栈和寄存器上下文，但由程序调用 `SwitchToFiber` 显式切换，内核完全不参与调度。程序在一个线程上创建两个 Fiber，主 Fiber 轮流切换它们。Fiber1 在第 1 步调用了 `Sleep(300)`：

d6_fiber_coro.exe —— 实测输出（Part1）

```
== Part1: Fiber（用户级线程，M:1）==
  Fiber0 第0步  tid=20764  &local=00000009D3FFFE00
  Fiber1 第0步  tid=20764  &local=00000009D40FFCF0
  Fiber0 第1步  tid=20764  &local=00000009D3FFFE00
  Fiber1 第1步  tid=20764  &local=00000009D40FFCF0
  Fiber1 调用 Sleep(300) —— 阻塞式系统调用，整个内核线程都睡了……
  ……300ms 内 Fiber0 一步也没走（耗时 307 ms）
  Fiber0 第2步  tid=20764  &local=00000009D3FFFE00
  Fiber1 第2步  tid=20764  &local=00000009D40FFCF0
```

三点值得注意：

1. **tid 始终相同**：两个 Fiber 都运行在同一个内核线程上，内核只看到这一个线程。
2. `&local` **不同，且每次切回来都不变**：每个 Fiber 有自己独立的栈（相距 1MB 左右），切换时保存和恢复栈指针，这和内核线程的栈私有是同一个原理，只是切换由用户代码完成。
3. **Fiber1 调用阻塞式系统调用后，Fiber0 整整 307ms 没有机会执行。**内核让这个线程睡眠，它不知道线程里还有一个 Fiber0 可以运行。

6.4 历史与现状：大家最后怎么选的

- **Linux**：早期的 LinuxThreads 实现有很多不符合 POSIX 的地方（例如每个线程 `getpid()` 返回不同值）。2003 年 NPTL 随 Linux 2.6 推出，采用 **1:1** 模型，配合内核 `clone` 线程组和 futex，线程创建和同步都很快。IBM 曾经开发过 M:N 的 NGPT，最终被放弃。
- **Solaris**：Solaris 8 之前是 M:N，Solaris 9 起改为 1:1，理由是 M:N 实现复杂、两层调度器互相干扰，而 1:1 在内核优化后已经足够快。
- **Windows**：内核线程 1:1；另外提供 Fiber（N:1 用户态原语）。Windows 7 引入过用户模式调度（UMS），支持 M:N，后来已被弃用。

- **Go**：goroutine 是 **M:N**。GMP 调度器：G 是 goroutine，M 是内核线程，P 是逻辑处理器（数量默认等于 CPU 核数）。它解决了"一个阻塞全部阻塞"：当 G 进入阻塞式系统调用时，M 与 P 分离，P 交给另一个 M 继续运行其他 G；网络 I/O 则由 netpoller（底层是 epoll/kqueue/IOCP）转成非阻塞，G 只是被挂起。Go 1.14 起还支持基于信号的异步抢占。
- **Java**：长期使用 1:1 的平台线程；**JDK 21 正式推出虚拟线程**（M:N），阻塞 I/O 时虚拟线程从载体线程上卸载。

✅ 面试答法：M:N 模型如何解决用户级线程的缺点

用户级线程有两个硬伤：**无法利用多核**和**一个阻塞全部阻塞**。M:N 用多个内核线程承载用户线程，解决了第一个；第二个则靠运行时**拦截阻塞操作**：网络 I/O 改用非阻塞 + 事件通知（epoll / IOCP），其他阻塞系统调用发生时把当前内核线程"让出去"、再启动或唤醒一个内核线程接管其余的用户线程。代价是运行时复杂度很高，而且对于运行时无法拦截的阻塞（比如 C 库里的阻塞调用、持有锁时的阻塞），仍然会占住内核线程。

本章小结

- 用户级线程：内核不可见，切换快、可定制，但不能用多核、一阻塞全阻塞。内核级线程：反之。
- 主流操作系统线程都是 1:1；M:N 的复杂度转移到了语言运行时（Go、Java 虚拟线程）。
- 实测：Fiber 上的 `Sleep(300)` 让同一线程上的另一个 Fiber 停了 307ms。

CHAPTER 07

线程、协程、任务

线程是操作系统的执行单元，协程是语言层面的执行单元，任务是"要做的一件事"本身。

本章目标

- 说清楚协程是什么，和线程在调度方式、栈、开销上的区别
- 区分有栈协程和无栈协程，理解 C++20 协程为什么是"无栈"的
- 理解"任务"是更高层的抽象，它可以跑在线程池上，也可以由协程表达

7.1 协程：可以暂停和恢复的函数

普通函数调用只有"调用"和"返回"两个动作，一旦返回，它的局部变量就没了。**协程**多了"挂起"和"恢复"：执行到一半可以暂停，把控制权交还给调用者，之后再从暂停的位置继续，局部变量都还在。

从这个意义上看，协程和第 6 章的用户级线程是一回事的两种叫法，侧重点不同：**用户级线程**强调"它是线程的一种实现"，**协程**强调"它是一种控制流结构"。两者共同的特征是：

- **协作式调度**：只在明确的挂起点（`co_await`、`yield`、`SwitchToFiber`）让出，不会在任意指令处被打断。
- **用户态切换**：不进入内核，代价是一次函数调用的量级。
- **内核不可见**：一个协程执行阻塞式系统调用，会阻塞它所在的整个线程。

7.2 有栈协程与无栈协程

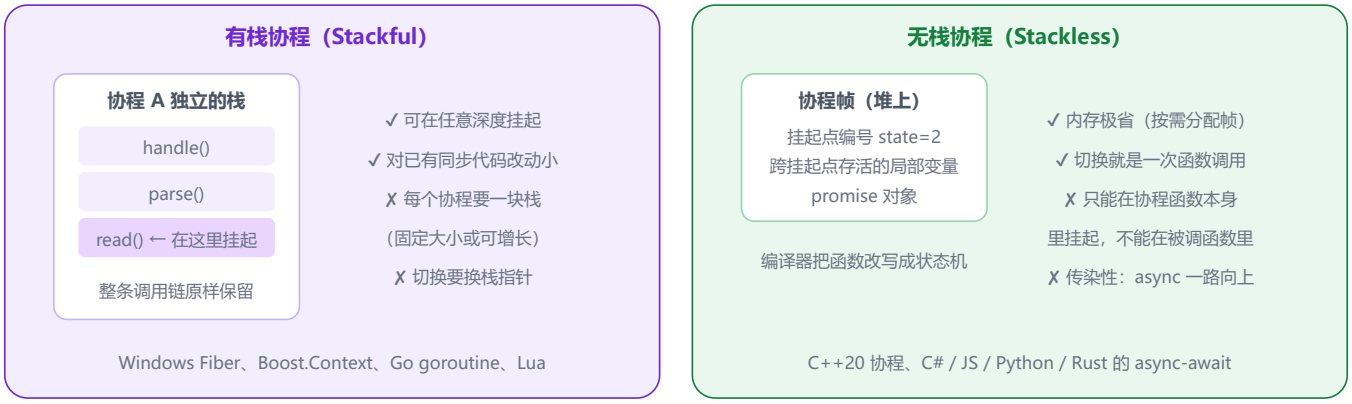


图 7-1 有栈协程保存整条调用栈；无栈协程只保存一个函数的状态，由编译器改写成状态机

d6 的 Part2 使用的是 C++20 协程。`Download` 函数里每次 `co_await Yield{}`，编译器会把它改写为大致这样的状态机：

C++20 协程被编译器改写后的等价逻辑（示意）

```
struct DownloadFrame {                  // 分配在堆上（编译器可能优化掉）
    int    state = 0;                   // 下一次恢复时从哪个挂起点继续
    const char* name; int chunks;       // 参数
    int    i;                           // 跨越 co_await 存活的局部变量
    Task::promise_type promise;
};
void resume(DownloadFrame* f) {
    switch (f->state) {
    case 0: for (f->i = 1; f->i ≤ f->chunks; ++f->i) {
                printf("[%s] 下载块 %d/%d\n", f->name, f->i, f->chunks);
                f->state = 1; g_ready.push_back(handle_of(f)); return;   // 挂起：直接 return
    case 1:     ;                                                        // 恢复：跳回循环里
            }
            printf("[%s] 完成\n", f->name);
            f->state = 2; return;                                        // final_suspend
    }
}
```

可以看到，**无栈协程挂起时不保留栈帧**，局部变量 `i` 被搬进了堆上的协程帧，恢复时用 `switch` 跳回去。这就是"无栈"的含义：它在运行时借用调用者线程的栈，挂起时什么都不留在栈上。

> ⚠️ **易错点：协程里同样有"栈上指针"的陷阱**
>
> 第 3 章讲过跨线程访问栈数据要注意生命周期，协程里也一样，而且更隐蔽：协程**按引用**接收的参数（`const std::string&`）在第一次挂起之后可能就失效了，因为调用者的临时对象在调用表达式结束时就被销毁。协程参数应该**按值**传递，让它被复制进协程帧。

7.3 任务：要做的那件事

任务（Task）不是一种执行机制，而是一个**工作单元的抽象**：一段要执行的计算，通常带有输入、结果和依赖关系。任务本身不规定由谁执行，可以：

- 提交到**线程池**，由某个工作线程取出执行（`std::async`、Intel TBB、Java `ExecutorService`、UE `TaskGraph`/`UE::Tasks`）；
- 由**协程**表达，在事件循环中被调度（C# `Task`、Python `asyncio.Task`、JS `Promise`）；
- 按依赖关系组成 **DAG**，由调度器尽量并行执行（游戏引擎的 Job System）。

> ⚠️ **术语冲突**
>
> Linux 内核源码里的 "task" 指 `task_struct`，也就是进程/线程，这和上面说的任务完全不同。面试中提到 task 时，最好先确认对方说的是哪个层面的概念。



图 7-2 任务、协程、线程处于不同的抽象层次，通常组合使用

7.4 一张总表

维度	进程	内核线程	有栈协程	无栈协程	任务
谁来调度	内核	内核	用户态运行时	用户态运行时 / 事件循环	线程池或事件循环
调度方式	抢占	抢占	协作（Go 有异步抢占）	协作	取决于执行者
地址空间	独立	共享	共享	共享	共享
栈	—	独立，1～8MB 虚拟	独立，KB 级起步	无，帧在堆上	借用执行线程的栈
切换开销	最大	约 1μs	约几十 ns	约几 ns～几十 ns	—
可利用多核	能	能	M:N 时能	需配合线程池	线程池能
数据竞争	无（不共享）	有	同线程内无，跨线程有	同上	有

⚠️ **易错点：单线程协程就没有并发问题了吗？**

单线程上的协程不会有**数据竞争**（同一时刻只有一个在运行，不存在两个线程同时写同一内存），但仍然有**逻辑上的竞态**：

跨越挂起点的检查-再执行

```
Task Withdraw(Account& acc, int amount) {
    if (acc.balance ≥ amount) {      // ① 检查：余额够
        co_await AuditLog(amount);   // ② 挂起！另一个 Withdraw 协程在这期间也通过了检查
        acc.balance -= amount;       // ③ 执行：两笔都扣了，余额变成负数
    }
}
```

每个 `co_await` 都是一个可能被插入其他逻辑的位置。**"检查"和"执行"之间如果有挂起点，就不是原子的。**

🎮 游戏开发视角

Unity 的 Coroutine（`yield return`）就是运行在主线程上的无栈协程，每帧由引擎恢复一次，所以它不能利用多核，也不能在里面执行耗时的同步计算，否则会卡帧。**UE 的 TaskGraph / UE::Tasks** 是任务系统，任务分发到工作线程上并行执行。**Naughty Dog 的引擎**在 GDC 演讲中介绍过基于 Fiber 的 Job System：每个 Job 运行在一个 Fiber 上，等待依赖时切换到别的 Fiber，内核线程数量固定等于核数，这样既有用户态切换的低开销，又能利用多核。

本章小结

- 协程是可以挂起和恢复的函数，协作式、用户态切换、内核不可见。
- 有栈协程保存整条调用栈，可在任意深度挂起；无栈协程由编译器改写成状态机，帧在堆上，只能在协程函数自身挂起。C++20 协程是无栈的。
- 任务是"工作单元"的抽象，与执行机制解耦，可以跑在线程池上，也可以用协程表达。
- 单线程协程没有数据竞争，但跨挂起点的"检查-执行"仍然有逻辑竞态。

PART FIVE

第五部分　平台补充：Linux 与 Windows【P1】

同样是"创建一个运行另一个程序的进程"，Linux 拆成 fork + exec 两步，Windows 用 CreateProcess 一步完成。两种设计各有道理，也各有坑。

CHAPTER 08

Linux：fork、exec、wait，孤儿与僵尸

fork 复制一个自己，exec 把自己换成别的程序，wait 给死去的子进程收尸。

本章目标

- 准确说出 fork 后哪些东西被复制、哪些被共享，理解写时复制
- 说出 exec 保留了什么、替换了什么
- 理解僵尸进程和孤儿进程的成因、危害和处理方法
- 能做 fork 计数类的笔试题，并解释 stdio 缓冲带来的"多打印"现象

8.1 Shell 是怎么执行一条命令的

在 bash 里输入 `ls -l`，bash 做了三件事：**fork** 出一个子进程（此时子进程还是 bash）；子进程调用 **exec** 把自己替换成 `/usr/bin/ls`；父进程 bash 调用 **wait** 等待子进程结束，拿到退出码（就是 `$?`），然后打印下一个提示符。

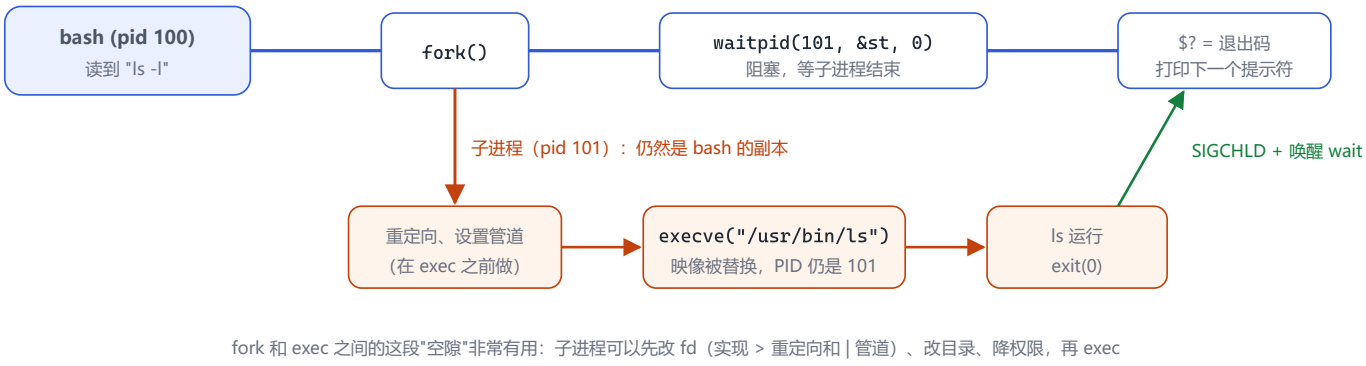


图 8-1 Shell 执行命令的经典三步：fork → exec → wait

8.2 fork：调用一次，返回两次

fork 的返回值

```
pid_t pid = fork();
if (pid < 0)       { /* 失败：进程数达到上限、内存不足 */ }
else if (pid == 0) { /* 子进程：fork 返回 0（可用 getppid() 找父亲） */ }
else               { /* 父进程：fork 返回子进程的 PID（父进程没有别的办法知道孩子是谁） */ }
```

fork 之后，父子进程从**同一个位置（fork 返回处）**继续执行，区别只在返回值。至于谁先运行，由调度器决定，**程序不能依赖任何顺序。**

fork 之后什么被复制，什么被共享

项目	fork 后	说明
地址空间（代码、数据、堆、栈）	**逻辑上复制，物理上写时复制**	见下文 COW
文件描述符表	**表复制，但指向同一个打开文件表项**	**文件偏移量和状态标志是共享的**！父子进程交替写同一个 fd，内容会接在一起而不是互相覆盖
信号处理方式、屏蔽字	复制	待处理信号集会被清空
当前目录、umask、uid/gid、资源限制	复制	
PID、PPID	**不同**	子进程获得新 PID，PPID 是父进程
文件锁（fcntl 记录锁）	**不继承**	
定时器、CPU 时间统计	**清零**	
线程	**只复制调用 fork 的那一个**	多线程程序 fork 的大坑，见下文

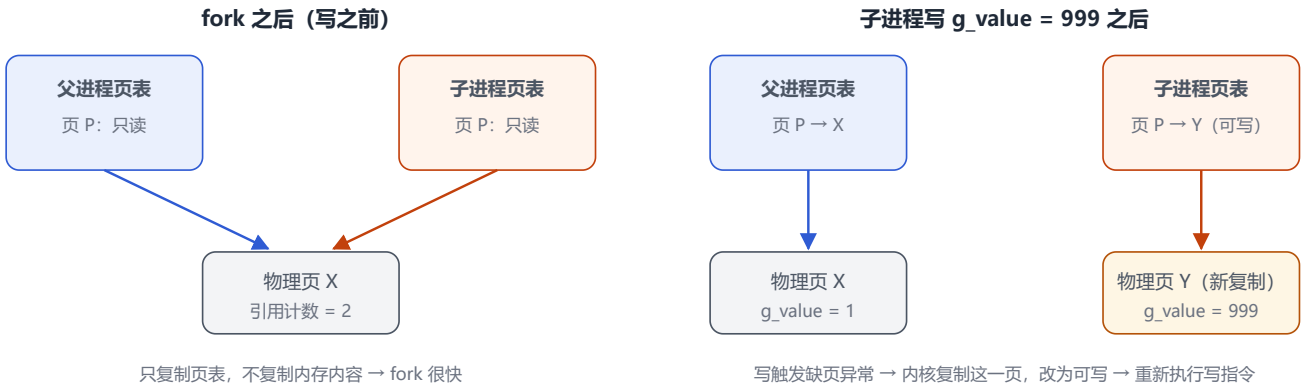


图 8-2 写时复制（Copy-On-Write）：只有被写的那一页才真正复制

> 🎮 **工程视角：COW 的妙用与代价**
>
> **Redis 的 RDB 持久化**就是利用 fork + COW：主进程 fork 出子进程，子进程看到的是 fork 那一刻内存的"快照"，慢慢写到磁盘；主进程继续处理写请求，只有被修改的页才会被复制。代价是：如果快照期间写入很多，被复制的页会很多，内存可能接近翻倍；另外 fork 本身要复制页表，几十 GB 的实例 fork 一次可能要几十到几百毫秒，期间主进程是阻塞的。开启透明大页（THP）还会让每次 COW 复制 2MB 而不是 4KB，所以 Redis 建议关闭 THP。

> ⚠️ **易错点：多线程程序里 fork**
>
> fork 只复制调用它的线程。假设 fork 时另一个线程恰好持有 `malloc` 内部的锁，子进程里这把锁仍然是"已锁定"状态，但持锁的线程在子进程中并不存在，**永远不会解锁**。子进程一调用 `malloc`（`printf` 内部也可能调用）就会死锁。因此 POSIX 规定：多线程程序 fork 后，子进程在调用 exec 之前**只能调用异步信号安全的函数**。实践建议：多线程程序要启动外部程序时，fork 后立刻 exec，或直接用 `posix_spawn`。`pthread_atfork` 可以注册在 fork 前后获取/释放锁的回调，但很难做到完备。

8.3 exec：换掉灵魂，保留身份

exec 系列函数把当前进程的程序映像**整个替换**为新程序：旧的代码、数据、堆、栈全部丢弃，按新程序的 ELF 文件重新建立地址空间，从新程序的入口开始执行。**exec 成功后不会返回**，因为调用它的代码已经不存在了。

exec 后保留	exec 后替换 / 重置
PID、PPID、进程组、会话	代码、数据、堆、栈、内存映射
文件描述符（**除非设置了** `FD_CLOEXEC`）	**自定义的信号处理函数恢复为默认**（函数地址在新程序里没有意义了）
当前目录、umask、资源限制	**除调用线程外的所有线程被销毁**
信号屏蔽字、待处理信号	用 `O_CLOEXEC` 打开的 fd 被关闭
被设为忽略（SIG_IGN）的信号仍然忽略	

exec 家族命名规律：`l`（list，参数逐个列出）/ `v`（vector，参数用数组）；`p`（在 PATH 中查找程序）；`e`（自己指定环境变量）。真正的系统调用只有 `execve`，其余都是 libc 包装。

> ✅ **实践建议：fd 泄漏与 O_CLOEXEC**
>
> 如果服务进程打开了数据库连接、监听 socket，fork + exec 启动一个脚本后，脚本进程会继承这些 fd。轻则资源泄漏，重则服务重启时端口一直被脚本进程占着。现代代码应该在打开 fd 时就加上 `O_CLOEXEC` / `SOCK_CLOEXEC`。

8.4 wait / waitpid：回收子进程

waitpid 用法要点

```
int status;
pid_t w = waitpid(pid, &status, 0);        // pid=-1 表示任意子进程；options=WNOHANG 表示不阻塞
if (WIFEXITED(status))   printf("正常退出，码=%d\n", WEXITSTATUS(status));
if (WIFSIGNALED(status)) printf("被信号 %d 杀死\n",  WTERMSIG(status));   // 例如 SIGSEGV=11
```

wait 有两个作用：**获取子进程的退出状态**，以及**让内核释放子进程残留的 task_struct**。第二个作用引出了僵尸进程。

8.5 完整示例：fork_exec_wait.c

demos/linux/fork_exec_wait.c

```c
int g_value = 1;
int main(void)
{
    printf("[parent] pid=%d  &g_value=%p\n", getpid(), (void*)&g_value);
    fflush(stdout);                     // 关键：fork 前清空 stdio 缓冲，否则缓冲区会被复制一份
    pid_t pid = fork();                 // 调用一次，返回两次
    if (pid == 0) {                     // ——— 子进程 ———
        g_value = 999;                  // 写时复制：此刻才真正复制这一页
        printf("[child ] pid=%d ppid=%d  &g_value=%p  g_value=%d\n",
               getpid(), getppid(), (void*)&g_value, g_value);
        fflush(stdout);
        execlp("ls", "ls", "-l", "/proc/self/exe", (char*)NULL);  // 用 ls 替换自身映像，PID 不
        perror("execlp");               // 只有 exec 失败才会走到这里
        _exit(127);
    }
    int status;
    pid_t w = waitpid(pid, &status, 0);  // 阻塞等待并回收，避免僵尸
    if (WIFEXITED(status))
        printf("[parent] 回收子进程 %d，退出码=%d，g_value 仍然=%d\n", w, WEXITSTATUS(status), g
    return 0;
}
```

Linux 典型输出（未在本机实测，PID 与地址会不同）

```
[parent] pid=4210  &g_value=0x55d0c3a4e010
[child ] pid=4211 ppid=4210  &g_value=0x55d0c3a4e010  g_value=999
lrwxrwxrwx 1 user user 0 Oct  8 12:00 /proc/self/exe → /usr/bin/ls
[parent] 回收子进程 4211，退出码=0，g_value 仍然=1
```

- 父子进程 `&g_value` 相同、值不同：与 Windows 的 d3 实测结论一致，同一虚拟地址、不同物理页。
- 第三行是 `ls` 打印的：`/proc/self/exe` 指向 `/usr/bin/ls`，说明 PID 4211 的程序映像已经被替换成了 ls。

8.6 僵尸进程

子进程退出时，内核释放它的大部分资源（内存、打开的文件），但**保留 `task_struct` 中的 PID、退出码和资源使用统计**，因为父进程可能还要通过 wait 读取它们。在父进程调用 wait 之前，这个只剩一

个"档案"的进程就是**僵尸进程**（Zombie），`ps` 中显示为 `Z`，命令名后带 `<defunct>`。

观察僵尸（demos/linux/zombie_orphan.c）

```
$ ./zo zombie &
child 5301 exit now
parent 5300 不调用 wait，睡 60s：此时子进程是僵尸
$ ps -o pid,ppid,stat,cmd --ppid 5300
  PID  PPID STAT CMD
 5301  5300 Z    [zo] <defunct>
$ kill -9 5301        # 无效：它已经死了，信号对僵尸没有意义
$ kill 5300           # 杀掉父进程 → 僵尸被 init 收养并立即回收
```

危害

单个僵尸几乎不占内存，但它**占用一个 PID**。PID 数量有上限（`/proc/sys/kernel/pid_max`，以及 cgroup 的 `pids.max`）。一个不断创建子进程又从不回收的服务，最终会让系统无法创建新进程，报 `fork: Resource temporarily unavailable`。

解决方法

1. **父进程同步 wait**：适合父进程本来就要等待子进程结束的场景（shell）。
2. **处理 SIGCHLD，在 handler 里循环 `waitpid(-1, NULL, WNOHANG)`**：适合父进程有自己的工作要做的场景（服务器）。**必须循环**，因为多个子进程同时退出时，标准信号不排队，多个 SIGCHLD 可能只被递送一次。
3. **显式设置 `signal(SIGCHLD, SIG_IGN)`**：POSIX 规定此时子进程退出后由内核自动回收，不产生僵尸（代价是父进程拿不到退出码）。
4. **两次 fork**：父进程 fork 出 A，A 再 fork 出 B 后立刻退出，父进程 wait 回收 A；B 成为孤儿，由 init 负责回收。传统守护进程的创建方法。

正确回收：SIGCHLD handler（zombie_orphan.c 的 reap 模式）

```c
static void on_sigchld(int sig)
{
    int saved = errno;                          // 信号处理函数可能打断主流程，保护 errno
    while (waitpid(-1, NULL, WNOHANG) > 0) {}   // 必须循环：多个 SIGCHLD 可能被合并成一个
    errno = saved;
}
```

8.7 孤儿进程

父进程先于子进程退出，子进程就成了**孤儿进程**。内核会把它**过继**给 init 进程（PID 1，现在通常是 systemd），或者过继给离它最近的、设置了 `PR_SET_CHILD_SUBREAPER` 的祖先进程（Linux 3.4 起支持）。新的父进程会负责 wait 它。所以**孤儿进程本身无害**，它会继续正常运行，退出后也会被回收。

观察孤儿（典型输出，未在本机实测）

```
$ ./zo orphan
child 5401: ppid=5400
parent 5400 exit, child becomes orphan
child 5401: ppid=1          # 或者是 systemd --user 等 subreaper 的 PID
child 5401: ppid=1
```

🔬 **深入：Docker 容器中的 PID 1 问题**

容器有独立的 PID 命名空间，容器里第一个进程是 PID 1，它会成为容器内所有孤儿进程的"收养者"。如果你的应用（例如一个 Node.js 或 Java 服务）作为 PID 1 运行，它通常不会去 wait 那些被过继给它的孤儿，结果它们退出后全部变成僵尸并不断累积。此外，PID 1 默认不会响应没有注册处理函数的 `SIGTERM`，导致 `docker stop` 要等超时后强杀。解决方法是用 `docker run --init` 或在镜像中用 **tini** / dumb-init 作为 PID 1，由它负责转发信号和回收子进程。

	僵尸进程	孤儿进程
成因	子先死，父没 wait	父先死，子还活着
状态	已终止，只剩 task_struct	正常运行
危害	占用 PID，累积后无法创建新进程	基本无害
谁来处理	父进程 wait；父进程死后由 init 回收	init / subreaper 收养
能否 kill	不能（已经死了），只能杀父进程	能

8.8 笔试题：fork 计数与缓冲区陷阱

经典题 1：会打印几个 "-"？

```
int main() {
    for (int i = 0; i < 2; ++i) {
        fork();
        printf("-");
    }
}
```

答案是 8 个，不是 6 个。逐步分析：

- i=0：1 个进程 fork 成 2 个，各执行一次 `printf("-")`。注意 `"-"` 没有换行，而且 stdout 输出到管道或文件时是全缓冲，所以这个 "-" **只是写进了各自的用户态缓冲区**，并没有真正输出。
- i=1：2 个进程 fork 成 4 个。**fork 会复制整个地址空间，包括 stdio 缓冲区**，所以 4 个进程的缓冲区里都已经有一个 "-"。每个进程再 printf 一次，缓冲区变成 "--"。
- 4 个进程退出时各自刷新缓冲区：4 × 2 = **8**。

如果改成 `printf("-\n")` 并输出到终端（终端上 stdout 是行缓冲），每次 printf 都会立即输出，答案就是 2 + 4 = **6**。这就是 8.5 节代码里 fork 前要 `fflush(stdout)`、子进程里要用 `_exit` 而不是 `exit` 的原因。

经典题 2：一共有几个进程（含原进程）？

```
fork() && fork() || fork();
```

▶ **点击查看答案：5 个**

本章小结

- **fork**：返回两次；地址空间写时复制；fd 表复制但共享文件偏移；只复制调用线程。
- **exec**：替换映像，保留 PID 和未设 CLOEXEC 的 fd；信号处理函数重置为默认；成功不返回。
- **wait**：获取退出状态并释放残留的 task_struct。
- **僵尸**：子先死父未收，占 PID，用 wait / SIGCHLD / SIG_IGN / 两次 fork 解决；**孤儿**：父先死，被 init 或 subreaper 收养，无害。

CHAPTER 09

Windows 的进程创建模型

没有 fork。CreateProcess 一步到位：建一个全新的进程，直接运行指定的程序。

本章目标

- 理解 CreateProcess 的参数和内部步骤
- 理解 Windows 的进程生命周期：内核对象、句柄与引用计数
- 对比 Windows 与 Linux 在父子关系、句柄继承、进程树管理上的差异

9.1 CreateProcess

d3_isolation.cpp 中的 RunChild（简化）

```
std::string cmd = "\"" + self + "\" child";        // CreateProcessW 要求命令行缓冲区可写
STARTUPINFOA si{ sizeof(si) };                     // 输入：窗口、标准输入输出句柄等
PROCESS_INFORMATION pi{};                          // 输出：两个句柄 + 两个 ID
CreateProcessA(nullptr, cmd.data(),
               nullptr, nullptr,                   // 进程 / 线程对象的安全属性
               FALSE,                              // bInheritHandles：是否继承可继承句柄
               0,                                  // 创建标志：CREATE_SUSPENDED、CREATE_NEW_CON
               nullptr, nullptr,                   // 环境变量、当前目录（nullptr = 继承父进程）
               &si, &pi);
WaitForSingleObject(pi.hProcess, INFINITE);        // 等待进程对象变为有信号（≈ waitpid）
GetExitCodeProcess(pi.hProcess, &code);            // 读退出码
CloseHandle(pi.hThread);                           // 两个句柄都必须关闭
CloseHandle(pi.hProcess);
```

内部发生了什么

根据 *Windows Internals* 的描述，CreateProcess 的主要阶段如下：



图 9-1 CreateProcess 的主要阶段

> ✅ **实践建议：CREATE_SUSPENDED 的用途**
>
> 先挂起创建，在子进程运行任何代码之前做一些设置：把它放进 Job 对象、调整优先级、注入调试或性能分析用的 DLL，然后再 `ResumeThread`。这在功能上相当于 Linux 中 fork 和 exec 之间的那段"空隙"。

9.2 进程是内核对象：句柄与引用计数

在 Windows 中，进程和线程都是**内核对象**，用户态通过**句柄**引用它们。内核对象有引用计数，只有当**所有句柄都关闭、且进程已经终止**时，`EPROCESS` 才会被真正释放。

- 进程终止（最后一个线程退出，或调用 `ExitProcess` / `TerminateProcess`）后，它的地址空间、句柄表等资源立刻被清理，进程对象变为**有信号**状态，等待它的 `WaitForSingleObject` 返回。
- 但只要还有人持有它的句柄，进程对象本身就保留着，里面记录着**退出码**，可以用 `GetExitCodeProcess` 读取。

> ⚠️ **易错点：Windows 上的"僵尸"**
>
> 这和 Linux 的僵尸进程本质相同：**进程已经结束，但为了让别人读取退出状态，内核对象被保留了下来。**区别是 Linux 依赖"父进程 wait"，Windows 依赖"所有句柄被关闭"。如果程序反复 `CreateProcess` 却忘了 `CloseHandle(pi.hProcess)` 和 `CloseHandle(pi.hThread)`，就会泄漏内核对象，在任务管理器里看到句柄数持续增长。这是 Windows 服务程序中很常见的泄漏。

9.3 父子关系很"松"

- **没有强制的进程树**：Windows 只在子进程中记录了创建者的 PID（`InheritedFromUniqueProcessId`），仅作参考。父进程退出不影响子进程，也不存在"过继给 init"的机制。**记录的父 PID 可能已经被别的进程复用**，所以不能依赖它。
- **不需要父进程来回收**：任何持有进程句柄的进程都可以等待它、读取它的退出码。
- **想要"父进程退出时子进程一起退出"**：要用 **Job 对象**。把子进程放进一个设置了 `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE` 的 Job，当 Job 的最后一个句柄关闭（例如父进程崩溃）时，Job 中的所有进程都会被终止。Chrome、VS 的构建系统、各种游戏启动器都用这个机制管理子进程。

9.4 句柄继承

Linux 子进程默认继承所有 fd（除非设置 CLOEXEC）；Windows 正好相反，**默认什么都不继承**，必须同时满足两个条件：句柄在创建时被标记为可继承（`SECURITY_ATTRIBUTES.bInheritHandle = TRUE` 或 `SetHandleInformation`），并且 `CreateProcess` 的 `bInheritHandles` 参数为 `TRUE`。继承来的句柄在子进程中**数值相同**，所以通常通过命令行参数或 `STARTUPINFO` 中的标准输入输出字段把句柄值告诉子进程。为了避免把无关的可继承句柄也带过去，可以用 `STARTUPINFOEX` + `PROC_THREAD_ATTRIBUTE_HANDLE_LIST` 精确指定要继承的句柄列表。

9.5 线程的创建

API	层次	说明
`CreateThread`	Win32	最底层的用户态 API。早期 CRT 中，用它创建的线程调用某些 CRT 函数可能导致 CRT 的每线程数据泄漏，所以过去推荐用 `_beginthreadex`；现代 UCRT 已经基本解决这个问题
`_beginthreadex`	C 运行时	在 CreateThread 之上初始化 CRT 的每线程数据
`std::thread` / `std::jthread`	C++ 标准库	MSVC 中最终调用 `_beginthreadex`；跨平台；`jthread` 析构自动 join 并支持停止令牌

默认栈大小：由 exe 的 PE 头决定（MSVC 链接器默认保留 1MB，可用 `/STACK` 修改），`CreateThread` 也可以单独指定。d1 实测每个线程栈 1024KB 正是这个默认值。

9.6 为什么 Windows 没有 fork

Windows NT 的设计受 VMS 影响，一开始就采用"创建进程时指定程序"的模型。fork 的语义（复制整个进程状态）与 Windows 的很多机制不容易兼容：大量内核对象句柄、GUI 资源、多线程（fork 只复制调用线程的问题）、DLL 的初始化状态。NT 内核其实**支持**类似 fork 的能力（早期 POSIX 子系统和 WSL1 都依赖它实现了 fork，通过内部的克隆进程接口），但 Win32 API 从未公开。

> 🔬 **深入：学术界对 fork 的反思**
>
> 2019 年 HotOS 会议上，微软研究院和波士顿大学的研究者发表论文 *A fork() in the road*，认为 fork 已经不适合现代操作系统：它与多线程不兼容；默认继承所有资源，带来安全隐患；写时复制需要过量提交内存；为了一个 exec 先复制整个地址空间的页表，在大内存进程上很慢。论文建议用 `posix_spawn` 这类"一步创建"的接口替代 fork + exec。这和 Windows CreateProcess 的设计理念一致。不过 fork 在"复制当前进程状态"的场景中仍然很有价值（如 Redis 快照、Android Zygote 预加载后 fork 出应用进程）。

9.7 Linux vs Windows 对照

维度	Linux	Windows
创建进程	fork（复制）+ exec（替换），两步	CreateProcess，一步
子进程初始内存	父进程副本（COW）	全新地址空间，加载指定 exe
资源继承	默认继承所有 fd，需 CLOEXEC 排除	默认不继承，需显式标记 + bInheritHandles
父子关系	严格的进程树，孤儿被 init 收养	只记录创建者 PID，无强制树
回收	父进程 wait，否则成为僵尸	关闭所有句柄后内核对象释放
等待结束	`waitpid`	`WaitForSingleObject(hProcess)`
进程组管理	进程组 / 会话 / cgroup	Job 对象
线程在内核中	与进程同为 task_struct	独立的 ETHREAD，进程只是容器
崩溃	SIGSEGV 终止整个进程	未处理异常终止整个进程（d3 实测 0xC0000005）

本章小结

- CreateProcess 一步建立全新进程并运行指定程序；需要在子进程运行前做设置时用 `CREATE_SUSPENDED`。
- 进程是内核对象，靠句柄引用计数管理生命周期；忘记 CloseHandle 会泄漏，效果类似僵尸。
- 父子关系松散，没有过继机制；要管理进程树，用 Job 对象。
- 句柄默认不继承，与 Linux 默认继承 fd 相反。

PART SIX

第六部分　实战选型与重点自测

把前面的知识用起来：什么时候选多进程，什么时候选多线程；然后用面试题检验自己。

CHAPTER 10

多进程还是多线程：选型方法与真实案例

选型本质上是在问：你更需要"隔离"，还是更需要"共享"？

本章目标

- 掌握一套多维度的选型方法，而不是只背"进程稳定、线程高效"
- 用实测数据感受创建和切换的开销差距
- 通过 Chrome、Nginx、数据库、游戏引擎等案例理解真实系统的取舍

10.1 实测：开销差多少

d4_cost.exe —— 实测输出（3 次运行，绑定单核）

```
创建+等待+销毁   线程：    103.0 us/次    进程：311603.1 us/次
创建+等待+销毁   线程：    122.7 us/次    进程：342084.0 us/次
创建+等待+销毁   线程：    112.8 us/次    进程：371013.6 us/次
单次切换(含事件系统调用)  线程间：0.68 / 0.69 / 0.69 us    进程间：1.50 / 1.24 / 1.55 us
```

⚠️ **如何解读这组数据**

进程创建测得约 **300ms 一次**，明显偏高。原因是本机装有企业安全软件，并且实验是在受监控的沙箱终端里运行的，**每次启动新进程都会被扫描和审计**。在一台干净的 Windows 机器上，CreateProcess 启动一个小程序通常在**几毫秒**量级；Linux 上 fork + exec 一个小程序通常在**几百微秒到一毫秒**量级；线程创建一般在**十几到几十微秒**量级（本实验绑定了单核并包含 join 等待，所以略高）。

不管环境怎样，结论都成立：**进程创建比线程创建贵 1～3 个数量级，跨进程切换比线程切换贵。**而在真实环境中，企业安全软件、杀毒软件让进程创建更贵，这也是工程上必须考虑的因素。

所以，"每来一个请求就创建一个进程"（早期 CGI 的做法）在高并发下不可行，需要**预先创建好进程池**（Nginx worker、PHP-FPM）；线程也应该用**线程池**复用，而不是每个任务新建一个。

10.2 选型的六个维度

维度	倾向多进程	倾向多线程
故障隔离	某个模块可能崩溃（第三方插件、不可信输入解析、渲染不可控网页），不能连累主程序	所有代码可控、质量统一
安全隔离	需要沙箱，限制某部分代码的权限（浏览器渲染进程）	同一信任域
数据共享	各单元基本独立，偶尔通信	**频繁共享大量数据**（游戏世界状态、内存缓存、大型矩阵）
创建 / 切换频率	长期运行、数量少	需要频繁创建和切换的轻量任务
扩展方式	未来要扩展到多台机器（进程间本来就用消息通信，换成网络更容易）	单机多核
语言 / 运行时限制	CPython 有 GIL，CPU 密集任务只能用多进程并行（3.13 起有实验性的无 GIL 构建）	C++ / Java / Go / Rust 线程可以真正并行

✅ **一句话原则**

默认用多线程（共享方便、开销小）；当"隔离"带来的价值超过"共享"带来的便利时，用多进程。真实系统几乎都是混合使用：多进程负责隔离和扩展，每个进程内部用多线程利用多核，再在线程上用事件循环或协程处理大量 I/O。

10.3 案例一：Chrome 为什么是多进程

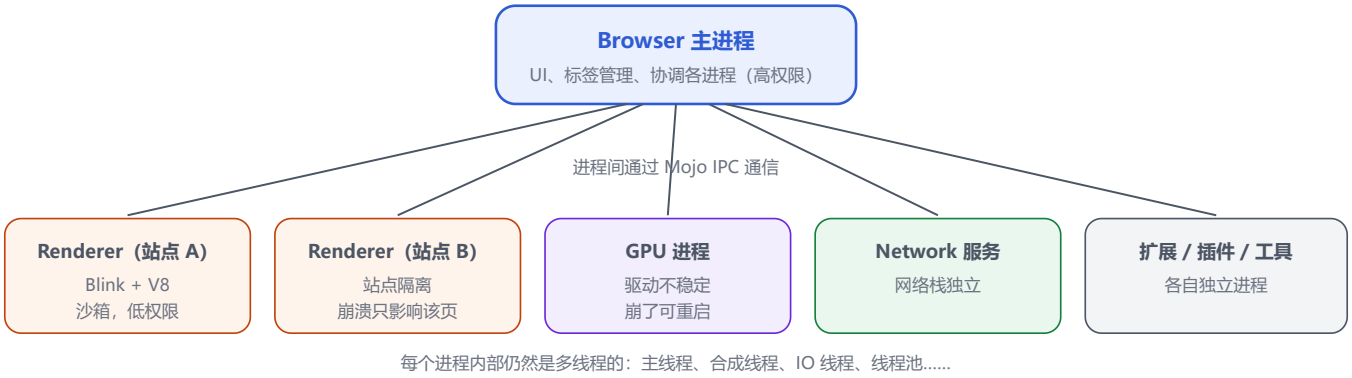


图 10-1 Chrome 的多进程架构（简化）

- **稳定性**：网页内容不可控，渲染引擎和 JS 引擎极其复杂。一个页面崩溃只显示"喔唷，崩溃啦"，不影响其他标签页。这正是第 3.7 节"崩溃半径"的应用。
- **安全**：渲染进程运行在沙箱中，没有文件系统和网络的直接访问权限。即使 V8 被攻破，攻击者还需要另一个漏洞才能逃出沙箱。2018 年 Spectre 漏洞让"同一地址空间内的数据可能被推测执行读到"成为现实，Chrome 因此全面启用了**站点隔离**，让不同站点一定运行在不同进程中。这也说明：**同进程内不存在真正的隔离**（呼应第 11 章自测题 2）。
- **代价**：内存占用大（每个进程都有自己的 V8 堆、Blink 数据），所以 Chrome 在内存紧张时会让同站点的标签共享进程。

10.4 案例二：Nginx 的 master-worker 多进程

- **master 进程**：读配置、绑定端口、管理 worker，不处理请求。
- **worker 进程**：数量通常等于 CPU 核数，每个 worker 是**单线程 + epoll 事件循环**，一个 worker 能同时处理上万个连接（并发不并行）。多个 worker 在多核上并行。
- **为什么用多进程而不是多线程**：worker 之间几乎不需要共享数据（每个连接独立处理），多进程带来了隔离：一个 worker 因第三方模块的 bug 崩溃，master 会立即拉起新的 worker。**热重载**也很自然：`nginx -s reload` 时 master 用新配置启动新 worker，旧 worker 处理完手上的连接后退出，服务不中断。
- **补充**：读磁盘文件可能阻塞事件循环，所以 Nginx 1.7.11 起提供了线程池（`aio threads`）专门处理阻塞的文件 I/O，又是一个混合使用的例子。

10.5 案例三：数据库的连接模型

	PostgreSQL	MySQL
模型	**每连接一个进程**（postmaster fork 出 backend 进程）	**每连接一个线程**（企业版 / Percona 等提供线程池）
共享数据	通过共享内存（shared buffers）在进程间共享缓存页	线程天然共享 buffer pool
优点	一个 backend 崩溃，postmaster 可以重置共享内存后恢复，隔离性好	连接开销小，可以支持更多连接
缺点	连接贵，大量连接时内存和切换开销大，生产中通常配合 PgBouncer 连接池	一个线程的内存破坏可能影响整个实例

两种模型都能做出优秀的数据库，说明选型没有绝对的对错。PostgreSQL 社区近年也在讨论向多线程模型迁移，主要动机就是降低连接开销。

10.6 案例四：游戏引擎与游戏服务器

🎮 游戏客户端：多线程为主

游戏世界状态（Actor、组件、物理、动画数据）在每一帧都要被逻辑、物理、动画、渲染频繁访问，共享量极大且对延迟敏感，跨进程通信无法承受，所以客户端以**多线程**为主。以 UE 为例：**GameThread** 执行游戏逻辑；**RenderThread** 把场景转换成渲染命令；**RHIThread** 把命令提交给图形 API；还有 **TaskGraph 工作线程**执行动画、物理、粒子等并行任务，以及音频线程、异步加载线程。帧与帧之间通过流水线让 GameThread 和 RenderThread 并行。

但也有例外：**Shader 编译**交给独立的 `ShaderCompileWorker` 进程。Shader 编译器是第三方的、可能崩溃，并且每个编译任务相互独立、几乎不共享数据，符合"隔离价值大于共享便利"的条件。编辑器侧还能用 Job 机制管理这些子进程的生命周期。

🎮 游戏服务器：多进程为骨架

大型在线游戏的服务端通常按功能或按房间拆成**多个进程**：网关、登录、匹配、战斗、聊天、存储代理等。对于 MOBA / 竞技类游戏，一场对局往往运行在战斗服务器进程（或进程中的一个独立房间）里。拆成多进程的理由是：**故障隔离**（一场对局出错不影响其他对局）、**横向扩展**（进程间本来就通过消息通信，部署到更多机器上很自然）、**独立发布**。进程内部则常见"单逻辑线程 + 若干网络 I/O 线程"的结构：逻辑单线程避免了复杂的锁，也让帧同步 / 状态同步的确定性更容易保证。

10.7 面试答法模板

✅ "什么时候选多进程，什么时候选多线程？"

1. **先说本质区别**：进程隔离资源，线程共享资源。这一点决定了两者的所有优缺点。
2. **多线程适合**：需要频繁共享大量数据、对通信延迟敏感、需要频繁创建和切换执行单元的场景，例如游戏客户端、内存缓存、并行计算。代价是需要同步，而且一个线程出错可能让整个进程崩溃。
3. **多进程适合**：需要故障隔离或安全隔离、各单元相对独立、需要分布到多台机器、或者语言运行时限制了多线程并行的场景，例如浏览器、Nginx worker、Shader 编译、Python 的 CPU 密集任务。代价是创建和切换开销大、通信要走 IPC。
4. **最后说混合**：真实系统通常是多进程 + 多线程 + 事件循环 / 协程分层组合。举一个自己熟悉的例子，比如 Chrome 或 Nginx。

本章小结

- 实测：进程创建比线程贵 1 ~ 3 个数量级（受安全软件影响可能更大），跨进程切换约为线程切换的 2 倍。
- 选型看六个维度：故障隔离、安全隔离、数据共享、创建切换频率、扩展方式、语言限制。
- Chrome、Nginx、PostgreSQL 选多进程是为了隔离；游戏客户端、MySQL 选多线程是为了共享；真实系统几乎都是混合的。

CHAPTER 11

重点自测与面试题库

先自己作答，再展开参考答案。两道重点题给出完整的答题结构。

11.1 重点题一：进程与线程有什么区别?

参考答案（按"定义 → 资源 → 开销 → 通信 → 健壮性 → 选型"组织）

1. **定义**：进程是资源分配和保护的基本单位；线程是 CPU 调度的基本单位。一个进程至少有一个线程，线程属于且只属于一个进程。
2. **资源**：进程拥有独立的虚拟地址空间、句柄 / 文件描述符表、安全凭据等；同一进程的线程共享这些资源，但各自维护寄存器上下文、栈、TLS、线程 ID 和调度状态。（可以引用 Microsoft Learn 的说法：同一进程内的线程共享虚拟地址空间和系统资源，但各自具有寄存器上下文、线程局部存储和栈等状态。）
3. **开销**：创建进程要建立地址空间和资源表，比创建线程贵 1 ~ 3 个数量级；切换进程要切换页表，导致 TLB 和缓存失效，比线程切换贵。（可以举自己实测的数据：同核切换线程 0.68μs、进程 1.50μs。）
4. **通信**：线程直接读写共享内存，但需要锁、原子操作等同步；进程间要通过管道、共享内存、socket 等 IPC。
5. **健壮性**：进程之间由 MMU 和独立页表强制隔离，一个进程崩溃不影响其他进程；线程共享地址空间，一个线程的非法访问会导致整个进程终止。
6. **补充（加分）**：Linux 内核里两者都是 `task_struct`，区别只在 `clone` 时共享哪些资源；Windows 内核中进程对象和线程对象是分开的，进程只是容器。

追问：什么时候选多进程，什么时候选多线程?

见 10.7 节的答题模板：**需要共享、追求效率选多线程；需要隔离、追求稳定和安全选多进程；实际系统多为混合。**结合 Chrome（隔离）、游戏客户端（共享）各举一例。

11.2 重点题二：线程有自己的栈，是否意味着其他线程不能通过指针访问其中的数据？

结论：不是。其他线程完全可以通过指针读写另一个线程栈上的数据。

1. **"栈私有"的真实含义**：每个线程分配到一块专用的地址区间来保存自己的调用链，编译器生成的代码只通过本线程的栈指针访问它。这是**分配上和约定上的私有**，不是访问权限上的私有。
2. **为什么能访问**：线程的用户栈位于所属进程的虚拟地址空间中（Microsoft Learn 原文：线程上下文包括"a user stack in the address space of the thread's process"），而同一进程的所有线程共用一套页表。MMU 只检查页面是否映射、是否可写，不区分是哪个线程在访问。
3. **实测证明**：d2_stack_ptr 中，线程 A 把栈上数组 `buf` 的地址交给线程 B；B 自己的栈范围是 `[...2121A00000, ...2121B00000)`，指针 `...21219FF7F0` 落在 A 的栈范围内，B 把数组每个元素乘以 100，A 随后读到 `{100, 200, 300, 400}`。
4. **真正的风险**：
   - **生命周期**：栈上数据随函数返回而失效。如果 A 的函数已经返回、或者 A 线程已经退出，B 再访问就是悬垂指针，读到垃圾或者写坏 A 后续的栈帧，且极难复现。
   - **数据竞争**：A、B 同时读写时，和共享全局变量一样需要同步。
5. **正确用法**：保证栈帧的生命周期覆盖所有使用者，例如在同一作用域内 `join`（或使用 `std::jthread`）、同步等待任务完成；否则就把数据放到堆上，用 `shared_ptr` 管理，或按值传递。
6. **对比（加分）**：不同进程之间才有硬件强制的隔离，另一个进程拿着同样的地址值也访问不到。即便如此，有权限的进程仍可以通过内核提供的接口（Windows 的 `ReadProcessMemory` / `WriteProcessMemory`，Linux 的 `ptrace`、`process_vm_readv`、`/proc/<pid>/mem`）访问其他进程的内存，调试器就是这么工作的，但这需要经过内核和权限检查。同理，`thread_local` 变量也只是"每线程一份副本"，取地址交给别的线程照样能访问。

11.3 更多自测题

▶ **1. 线程共享进程的哪些资源？各自独有哪些？**

▶ 2. 一个线程崩溃会导致整个进程崩溃吗？为什么？有没有例外？

▶ 3. 就绪和阻塞有什么区别？为什么阻塞不能直接变为运行？

▶ 4. 进程切换为什么比线程切换慢？

▶ 5. 什么是僵尸进程？如何避免？kill -9 能杀死僵尸吗？

▶ 6. fork 之后父子进程共享文件偏移吗？

▶ 7. 多线程程序中调用 fork 有什么风险？

▶ 8. exec 之后，原来打开的文件还在吗？信号处理函数呢？

▶ 9. 用户级线程和内核级线程的区别？M:N 模型解决了什么问题？

▶ 10. 协程和线程有什么区别？有栈协程和无栈协程呢？

▶ 11. errno 是全局变量，多线程下为什么不会互相干扰？

▶ 12. 线程栈默认多大？如何避免栈溢出？

▶ 13. 下面代码有什么问题？`void f(){ int x = 0; std::thread t([&]{ x++; }); t.detach(); }`

▶ 14. Windows 上父进程退出，子进程会怎样？如何让子进程跟随父进程退出？

▶ 15. 单核 CPU 上多线程有意义吗？

▶ 16. volatile 能保证多线程下 counter++ 正确吗？

APPENDIX

附录 • 考前速记卡

面试前 10 分钟过一遍。

---

① 三者关系

程序 = 静态文件；进程 = 资源容器（分配与保护单位）；线程 = 执行流（调度单位）。1 个程序 → N 个进程；1 个进程 → ≥1 个线程。

② 并发 vs 并行

并发 = 结构上重叠推进（单核也行）；并行 = 物理上同时执行（需多核）。Amdahl：上限 1/(1−p)。

③ 共享

地址空间（代码、全局、堆、映射、各线程栈所在区域）、fd / 句柄表、凭据、cwd、信号处置、PID。

④ 独立

TID、寄存器（PC/SP）、用户栈 + 内核栈、TLS（errno）、调度状态 / 优先级、信号屏蔽字、SEH 链。

⑤ 栈私有 ≠ 不可访问

同一地址空间 + 同一套页表 → 拿到指针就能读写。风险在生命周期和数据竞争。进程间才有硬件隔离。

⑥ 五状态

新建 → 就绪 ⇄ 运行 → 终止；运行 → 阻塞 → 就绪。没有阻塞 → 运行、就绪 → 阻塞。就绪差 CPU，阻塞差事件。

⑦ Linux 状态

R（就绪 + 运行）、S（可中断睡眠）、D（不可中断，kill -9 无效）、T（停止）、Z（僵尸）。load = R + D。

⑧ 上下文切换

保存/恢复寄存器、换内核栈和 TLS 基址；跨进程还要换页表 → TLB/缓存失效。实测 0.68μs vs 1.50μs。

⑨ PCB / TCB

PCB：标识、内存、资源、家族、统计。TCB：上下文、调度信息、栈。Linux task_struct 用 mm/files 指针表达共享。

⑩ ULT / KLT

关键：内核是否可见。ULT 快但不能多核、一阻塞全阻塞；KLT 反之。1:1（Linux/Windows）、N:1（Fiber）、M:N（Go/虚拟线程）。

⑪ 协程 / 任务

协程：用户态、协作式、可挂起的函数；有栈（Fiber、goroutine）vs 无栈（C++20，状态机 + 堆帧）。任务：工作单元抽象，跑在线程池或协程上。

⑫ fork

返回两次（子 0，父 子PID）；COW；fd 表复制但共享偏移；只复制调用线程；fork 前 fflush。

⑬ exec / wait

exec 换映像保 PID 和非 CLOEXEC 的 fd，handler 重置，成功不返回。wait 取退出码并释放 task_struct。

⑭ 僵尸 / 孤儿

僵尸：子死父未收，占 PID，kill 不掉 → wait / SIGCHLD 循环 / SIG_IGN / 两次 fork。孤儿：父先死 → init/subreaper 收养，无害。

⑮ Windows

CreateProcess 一步建新进程；CREATE_SUSPENDED 先设置后运行；句柄引用计数，忘关 = 泄漏；无进程树，用 Job 管理；句柄默认不继承。

⑯ 选型

默认多线程（共享、高效）；隔离价值更大时多进程（稳定、安全、扩展、GIL）。真实系统 = 多进程 × 多线程 × 事件循环/协程。

参考资料

- Microsoft Learn：*About Processes and Threads*、*Creating Processes*、*Thread Stack Size*、*Fibers*、*Job Objects*。
- Pavel Yosifovich 等，*Windows Internals, 7th Edition, Part 1*，第 3 章（进程与 Job）、第 4 章（线程）。
- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau，*Operating Systems: Three Easy Pieces*，进程 API、线程、并发相关章节。
- W. Richard Stevens, Stephen A. Rago，*Advanced Programming in the UNIX Environment, 3rd Edition*，第 8 章（进程控制）、第 11 ~ 12 章（线程）。
- Linux man-pages：`fork(2)`、`execve(2)`、`wait(2)`、`clone(2)`、`prctl(2)`、`pthreads(7)`。
- Andrew Baumann et al., *A fork() in the road*, HotOS 2019。

- Christian Gyrling, *Parallelizing the Naughty Dog Engine Using Fibers*, GDC 2015。
